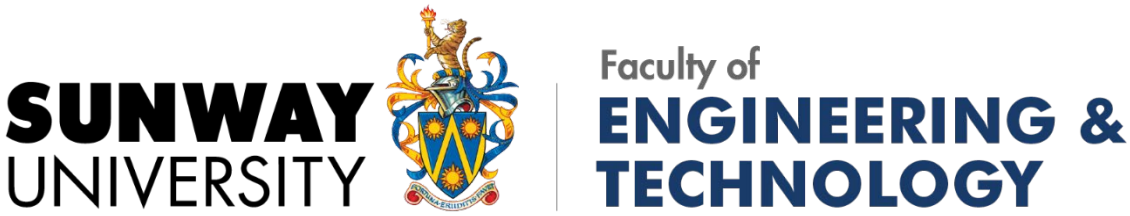


CAPSTONE PROJECT 2

Final Report

Personalized Restaurant Ordering System

by

YEAP CHAN LEONG

22049837

Bachelor of Science (Honours) in Computer Science

Supervisor: Professor Serge Demidenko

Semester: April 2026

Date: 2 August, 2026

School of Computing and Artificial Intelligence

Faculty of Engineering and Technology

Sunway University

Table of Contents

Abstract

This study presents a Personalized Restaurant Ordering System aimed at small restaurants that lack large volumes of historical order data. The system provides personalized, explainable dish recommendations while supporting the full customer ordering workflow. Besides, it is a mobile-first web application built using Rust, Axum, Tokio, server-rendered HTML, JavaScript and CSS, with data persisted in local CSV and JSONL files. Moreover, the recommendation engine draws on ingredient preferences, dish tags, historical co-order relationships, popularity and time context. Disliked ingredients are treated as strict exclusions, and an adaptive weighting scheme adjusts how much each signal contributes depending on the evidence available. Functional testing showed that the prototype covered menu browsing, preference input, personalized recommendations, meal-set generation, cart management, checkout, order tracking, and administrator tools. Controlled experiments further illustrated how ingredient preferences and simulated co-order data shift recommendation rankings. Every result was deterministic and came with an explanation, component scores, and indicators of the supporting evidence. However, the study is constrained by a small historical dataset, controlled testing conditions, local file-based storage and a single-restaurant prototype scope. The evaluation demonstrates system behaviour and technical feasibility rather than generalizable prediction accuracy or customer satisfaction in live settings. Therefore, the prototype shows how lightweight and transparent recommenders can be embedded in a practical restaurant ordering system without relying on large databases, external language models or heavyweight machine-learning frameworks.

Chapter 1: Introduction

1.1 Research Background

Digital restaurant ordering systems have increasingly replaced printed menus by allowing customers to browse dishes and place orders through websites, mobile applications, self-service kiosks or QR codes. These systems can improve convenience and reduce reliance on physical menus. However, many digital menus function mainly as electronic catalogues where dishes are arranged by category or general popularity without adapting to individual customer preferences.

Selecting a meal can become difficult when customers are presented with many dishes but receive limited guidance about which choices match their tastes. Food recommender systems address this problem by identifying suitable dishes based on information such as ingredients, food categories, dietary preferences, previous orders, and rating factors. Research has shown that food recommendation is a specialized area because food choices are affected by personal taste, dietary restrictions, health considerations, context, and available menu information (Trattner & Elsweiler, 2017; Li et al., 2018).

Common recommendation approaches include:

- **Content-based filtering,** which recommends items based on their attributes and their similarity to a user's preferences;
- **Collaborative filtering,** which identifies patterns from the behaviour of users or relationships among items; and
- **Hybrid recommendation,** which combines multiple recommendation methods to reduce the weaknesses of using a single method.

Recent food recommendation research has combined content attributes, collaborative information, contextual factors and association patterns to improve personalization. Studies have also explored restaurant recommendations based on ratings, taste profiles, ingredients, dietary types and frequently purchased combinations (Munaji & Emanuel, 2021; Singh & Dwivedi, 2023; Sharma et al., 2023). Nevertheless, many of these approaches assume that sufficient ratings, reviews, user histories, or interaction records are available.

This assumption does not reflect the small and single restaurant. Compared with large food-delivery platforms, an individual restaurant typically has:

- fewer customers and completed orders;
- little or no explicit rating information;
- fewer repeated interactions from identifiable users;
- limited technical infrastructure;
- limited resources for maintaining complex machine-learning pipelines; and
- a need for recommendations that staff and customers can understand.

Collaborative methods can become unreliable when interaction data are sparse. For example, a dish pair appearing together only once may appear to have a strong relationship even though the available evidence is weak. Besides, explainability is another important consideration. A recommendation score alone does not tell the customer why a dish was suggested or whether the recommendation is supported by substantial evidence. Explainable recommender research emphasizes that systems should provide understandable reasons alongside recommendation results rather than producing only unexplained rankings (Zhang & Chen, 2020).

This project addresses these issues through a **Personalized Restaurant Ordering System** designed for a small restaurant environment. It is implemented as a mobile-first web application so that customers can access the ordering interface from their own phones instead of requiring a dedicated tablet at every table. The recommendation engine uses transparent and deterministic rules combining:

- ingredient preferences;
- preferred dish tags;
- disliked-ingredient exclusions;
- historical co-order evidence;
- dish popularity; and
- time-based context.

1.2 Problem Statement

The problem to be addressed through this study is the lack of a lightweight personalized recommendation mechanism tailored specifically for individual restaurant ordering systems. When every customer is shown the same menu, the system may offer limited assistance in identifying dishes that match individual tastes, preferred ingredients, dietary preferences, or current meal choices. Previous restaurant recommendation studies have therefore explored the use of food attributes, customer preferences, ratings, and ordering behaviour to support more relevant food selection (Li et al., 2018; Gunawardena and Sarathchandra, 2020; Bondevik et al., 2024).

A major difficulty is that many conventional recommender systems rely on extensive interaction data. Collaborative filtering typically derives recommendations from user ratings, purchase records, or similarities among users and items. Munaji and Emanuel (2021), for example, developed a restaurant recommendation approach using user ratings and Pearson correlation. Although this approach can identify preference similarities, it depends on users providing sufficient ratings. Food recommendation studies using collaborative filtering similarly require user–item interactions from which meaningful patterns can be calculated (Dwivedi, 2023; Sharma et al., 2023).

This shows a problem in a single-restaurant environment. A small restaurant may have relatively few historical orders, no explicit dish ratings and limited repeated interaction from identifiable customers. Under these conditions, the available user–item or dish–dish data are sparse. New customers also have no personal ordering history, which creates a customer cold-start problem. Consequently, a recommendation method that relies entirely on collaborative data may fail to produce useful results or may assign excessive importance to relationships supported by only a small number of orders. Reviews of food recommender systems continue to identify data sparsity, cold start, limited datasets and evaluation quality as important challenges within the field (Chow et al., 2023; Bondevik et al., 2024).

Content-based recommendation provides a possible response to this limitation because it uses item attributes rather than depending entirely on previous user interactions. In the food domain, dishes can be represented using ingredients, cuisine types, categories, dietary labels, prices and other food characteristics. These attributes allow recommendations to be generated from a customer's explicitly stated preferences even when no personal order history is available (Li et

al., 2018; Dwivedi, 2023). However, content-based methods alone may overemphasize dishes with similar attributes and cannot fully capture behavioural relationships such as which dishes are commonly ordered together.

Moreover, hybrid recommendation approaches combine content and collaborative signals to address the weaknesses of individual methods. Recent food recommendation systems have combined user preferences, food characteristics, taste profiles, ratings and collaborative information to improve the range and relevance of recommendations (Keshav et al., 2023; Sharma et al., 2023; Bondevik et al., 2024). Nevertheless, hybrid systems introduce an additional design problem of balancing the contribution of each recommendation component. A fixed weighting may be inappropriate when one customer provides detailed preferences but no selected dishes while another provides selected dishes with strong co-order evidence but no explicit ingredient preferences.

The problem addressed by this research can therefore be summarized through four challenges:

1. **Limited historical data:** A single restaurant may not possess enough ratings or order records to support conventional collaborative filtering reliably.
2. **Customer cold start:** New customers require recommendations even though they have no previous interaction history.
3. **Preference and restriction handling:** Explicitly disliked ingredients must not be overridden by popularity or co-order evidence.
4. **Lack of transparency:** Customers and administrators need to understand which signals influenced a recommendation and how much supporting evidence is available.

To address these challenges, this project develops a lightweight, mobile-first restaurant ordering system that combines ingredient preferences, preferred tags, historical co-order evidence, dish popularity, and time context. Dishes containing disliked ingredients are removed before ranking, while adaptive weights adjust the contribution of the available recommendation signals. The system also presents recommendation explanations, component scores, and evidence indicators rather than treating the output as an unexplained prediction. The implementation uses deterministic decision rules instead of a heavyweight machine-learning framework, making it suitable for controlled evaluation within a limited-data, single-restaurant prototype.

1.3 Research Questions

Based on the problems identified above, this study addresses the following research questions:

> **RQ1:** How do liked and disliked ingredient preferences affect dish ranking and restriction compliance in a single-restaurant recommendation system?
>
> **RQ2:** How does increasing co-order evidence between an anchor dish and a candidate dish affect association measures and recommendation ranking under limited-data conditions?
>
> **RQ3:** How do ingredient-only, co-order-only, and hybrid recommendation methods compare in recovering a hidden dish from historical order baskets?

The research questions focus on observable recommendation behaviour. They do not assume that the prototype automatically improves customer satisfaction, commercial performance, or general prediction accuracy.

1.4 Research Aim and Objectives

1.4.1 Research Aim

This research aims to design, implement and evaluate a lightweight and explainable Personalized Restaurant Ordering System that provides personalized dish recommendations within a limited-data and single-restaurant environment.

1.4.2 Research Objectives

The objectives of this research are:

RO1: To develop a mobile-first web-based restaurant ordering system that supports menu browsing, search, preference selection, personalized recommendations, cart management, checkout, order tracking and administrator management.

RO2: To implement an ingredient-based recommendation component that rewards liked ingredients and preferred dish tags while treating disliked ingredients as hard exclusions.

RO3: To implement an item-to-item co-order component that derives behavioural relationships from historical order baskets without requiring explicit customer ratings.

RO4: To integrate ingredient matching, co-order evidence, popularity and time context through an adaptive and explainable hybrid scoring approach.

RO5: To develop controlled, repeatable and non-destructive experiments for analysing ingredient impact, co-order impact and differences among ingredient-only, co-order-only, and hybrid recommendation methods.

RO6: To evaluate the recommendation outputs using measures such as Hit@K, hidden-dish rank, preference match rate, restriction violations, support, association confidence, lift, component scores, and evidence confidence.

1.5 Project Scope and Limitations

1.5.1 Project Scope

The project covers three principal system areas.

A. Customer-side Functions

The customer interface supports:

- temporary restaurant-session registration;
- mobile-responsive menu browsing;
- category-based navigation;
- menu search and dish location;
- selection of liked ingredients;
- selection of disliked ingredients;
- selection of preferred tags and context dishes;
- personalized dish recommendations;
- recommendation reasons and evidence indicators;
- Familiar, Balanced, and Discover diversity modes;
- budget-aware meal-set generation;
- cart and quantity management;
- checkout; and
- order-status tracking.

B. Administrator-side Functions

The administrator's interface supports:

- separately scoped administrator authentication;
- dashboard summaries;
- viewing live and historical orders;
- updating order statuses;
- creating, editing, deleting, and disabling dishes;

- exporting menu information;
- viewing dish-popularity and co-order information;
- inspecting production recommendation behaviour; and
- conducting controlled recommendation experiments.

C. Recommendation-system functions

The recommendation engine includes:

Function	Purpose
Ingredient matching	Measures alignment with liked ingredients and preferred tags
Hard exclusion	Removes unavailable, selected, or disliked-ingredient dishes
Co-order analysis	Identifies dishes that appear together in historical baskets
Popularity scoring	Provides a deterministic fallback when other evidence is limited
Time-context scoring	Introducing simple time-based business context
Adaptive weighting	Adjust component influence according to available evidence
Diversity reranking	Balances familiarity and menu discovery
Explanation generation	States why each dish was recommended
Evidence confidence	Indicates the strength of supporting recommendation evidence
Meal-set generation	Suggests combinations according to budget and restrictions

The current implementation is a browser-based Rust application using Axum, Tokio, server-rendered HTML, plain JavaScript, mobile-first CSS, and local CSV/JSONL persistence.

1.5.2 Scope Exclusion

The following functions are outside the scope of the prototype:

- online payment processing;
- certified allergy or nutritional assessment;
- multi-restaurant management;
- production-scale cloud deployment;

- permanent cross-visit customer profiles;
- statistically trained deep-learning models; and

1.5.3 Research Limitations

The prototype is subject to the following limitations:

1. **Small historical dataset:** The prototype uses a limited number of historical order baskets. The identified co-order relationships may therefore not represent broader customer behaviour.
2. **Single-restaurant environment:** The design is evaluated using one menu and one restaurant context. Its results may not transfer directly to restaurants with substantially different cuisines, menu sizes, or customer populations.
3. **Controlled and synthetic evaluation:** Simulated co-orders and hidden-dish tests demonstrate how the recommendation rules behave. They do not prove performance in a long-term commercial deployment.
4. **No large-scale user study:** The research does not establish that the recommendations improve customer satisfaction, decision time, sales revenue or customer retention.
5. **Evidence confidence is not predictive probability:** The confidence indicator represents the amount and quality of evidence supporting a recommendation. It does not measure the probability that the customer will like or purchase the dish.

1.6 Significance of the Study

1.6.1 Significance to Customers

The project provides customers with a more guided alternative to browsing a uniform digital menu. Instead of depending only on dish popularity, customers can state which ingredients they like or dislike and indicate the dishes relevant to their current selection. Based on previous research, it has been shown that attributes such as ingredients, taste, dietary type and previous choices can contribute useful information for personalized food selection (Li et al., 2018; Singh and Dwivedi, 2023; Sharma et al., 2023). The system also maintains customer control. Recommendations do not replace the full menu, and customers are free to reject the suggestions. Disliked ingredients operate as hard exclusions so that a highly popular dish cannot re-enter the recommendation list merely because it has strong behavioural evidence.

1.6.2 Significance to Restaurant Administrators

For admin, the project shows that personalized recommendation can be integrated with an ordering system without requiring the costly data infrastructure. For example, earlier restaurant systems such as Best Dish also demonstrate the practical of combining digital menus with food-item recommendations (Gunawardena and Sarathchandra, 2020). The system allows complete orders to contribute to future popularity and co-order pattern. Admin can see popular dishes, commonly ordered pairs, recommendation component scores and the historical evidence behind suggestions. This provides a more transparent basis for understanding recommendation behaviour than an interface that displays only a final ranked list.

1.6.3 Academic Significance

This study has contributed to applying recommender-system principles to restaurants, which are characterized as having limited data and operating in a single-restaurant setting. Moreover, food recommender research covers a wide variety of techniques, datasets, objectives and evaluation approaches, but data availability, cold start, evaluation quality, and the specialized nature of food choice remain continuing challenges (Trattner and Elsweiler, 2017; Chow et al., 2023; Bondevik et al., 2024).

This project contributes through the combined investigation of:

- explicit ingredient preferences;
- hard dislike constraints;
- implicit co-order behaviour;
- adaptive recommendation weights;
- evidence strength;
- recommendation explanations;
- diversity reranking; and
- controlled non-destructive experiments.

Last but not least, this project also distinguishes between **ranking score** and **evidence confidence**. This distinction prevents a dish with a high rule-based score from automatically being described as having strong historical support. The presentation of explanations and evidence is consistent with explainable recommendation research which aims to make recommendation outputs understandable to users and system designers (Zhang and Chen, 2020).

1.6.4 Sustainability Considerations

The system has potential relation to **Sustainable Development Goal 9: Industry, Innovation and Infrastructure** especially through the development of an accessible digital solution intended for a small-business environment. Goal 9 includes infrastructure, industrialization, technological capability and innovation.

Chapter 2: Literature Review

2.1 Introduction

Recommendation systems are widely used on digital platforms to support user decision-making by filtering available options according to preferences and previous interactions. In a restaurant environment, customers may need to compare many dishes before identifying a suitable choice. Recommendation systems can support this process by narrowing the available options and highlighting items that correspond with the customer's preferences or ordering context (Munaji & Emanuel, 2021).

However, applying recommendation methods in a single-restaurant environment can be challenging because explicit ratings, permanent customer profiles and repeated interaction histories may be limited. Many conventional approaches assume that sufficient behavioural data are available for identifying reliable relationships among users or items. This assumption may not reflect the conditions of a small restaurant, where historical orders are fewer and new customers begin without any personal interaction history. Therefore, it is necessary to examine recommendation methods that can operate with sparse data while remaining practical and understandable.

This chapter reviews the recommendation approaches relevant to the present study and evaluates their strengths, limitations and suitability for a limited-data restaurant setting. First, collaborative filtering and co-order-based recommendation are discussed, followed by ingredient-based and hybrid approaches. Advanced and multimodal models are then considered to clarify the trade-off between predictive complexity and practical deployment. In addition, the chapter reviews explainable recommendation used to assess association behaviour. Finally, the reviewed literature is synthesised to identify the research gap addressed by the project.

2.2 Collaborative Filtering and Co-Order Recommendation

Collaborative filtering is a widely applied recommendation approach that identifies patterns from previous user interactions. User-based collaborative filtering recommends items based on similarities among users, whereas item-based collaborative filtering identifies

relationships among items that have received similar interactions. In restaurant systems, these interactions may include explicit ratings, purchase histories or dishes appearing together in the same order basket (Sharma et al., 2023).

Previous restaurant recommendation studies have applied both user-based and item-based collaborative filtering. Munaji and Emanuel (2021) used Pearson correlation to identify similarities among users from explicit dish ratings. Their findings indicate that rating-based collaborative filtering can generate relevant recommendations when sufficient feedback is available. By contrast, Permana (2024) compared user-based and item-based approaches and found that both could support restaurant recommendation under the evaluated conditions. The item-based approach is particularly relevant to the present study because it focuses on relationships among dishes rather than requiring permanent customer profiles.

However, the effectiveness of collaborative filtering depends strongly on the amount of interaction data available. In a single-restaurant environment, customers may complete orders without rating individual dishes, while less popular dishes may appear in only a small number of transactions. Consequently, the user–item or item–item interaction data may be too sparse to support reliable similarity estimates (Sharma et al., 2023). New customers and newly introduced dishes also face cold-start conditions because no previous interactions are available for them.

One way to reduce dependence on explicit ratings is to use completed order baskets as implicit feedback. When two dishes repeatedly appear in the same basket, their co-occurrence can be interpreted as evidence of an item-to-item relationship. This approach is suitable for restaurant ordering because the required data are generated through normal transactions. Nevertheless, co-order evidence does not remove the sparsity problem entirely, since relationships derived from only one or two baskets may still be weak or unstable.

For this reason, collaborative evidence can be supplemented with content information such as ingredients, categories or dish tags. Hybrid approaches combine behavioural and item-level information so that content attributes can support recommendations when co-order evidence is limited (Singh & Dwivedi, 2023). In the present study, ingredient matching is combined with co-order-based item-to-item recommendation to represent both explicit customer preferences and implicit dish relationships.

2.3 Ingredient-Based Matching

Ingredient-based recommendation uses the recorded components of a dish as content information for matching menu items with customer preferences. Unlike behavioural evidence, ingredient metadata does not depend on customers having previously rated or ordered a dish. It can therefore provide a recommendation signal when interaction data are unavailable or sparse. Ingredient information may also support understandable recommendation reasons because the system can identify which recorded components correspond with the customer's stated preferences.

Ingredient information can be represented as structured lists, tags or feature vectors. Vector-based representations allow similarities between dishes to be calculated according to the ingredients they share, while simpler matching approaches compare recorded ingredients directly with customer preferences. Some methods use weighting techniques such as Term Frequency–Inverse Document Frequency to give greater importance to ingredients that are more distinctive across the available items. Shinagam (2025) reported that content-based information remained useful under sparse-data conditions, suggesting that item attributes can provide an alternative recommendation signal when interaction evidence is limited.

Ingredient-based recommendation offers several advantages in a limited-data environment. It can reduce the new-item cold-start problem because a dish can be compared with customer preferences before it has accumulated historical interactions. It can also support transparent explanations by identifying the ingredients or tags that contributed to a recommendation. Nevertheless, the quality of the result depends on the completeness and consistency of the menu metadata. Ingredient presence alone does not represent quantity, flavour intensity, preparation method or the interaction among ingredients. Two dishes may therefore share ingredients while providing substantially different dining experiences.

Ingredient similarity also cannot reveal every behavioural relationship between dishes. Items that share few ingredients may still be complementary because customers frequently order them together. Ingredient information is therefore useful for representing stated preferences, but it does not fully replace collaborative evidence derived from historical orders. In the present study,

ingredient matching is combined with item-to-item co-order evidence so that the recommendation system can consider both explicit customer preferences and implicit relationships among dishes.

2.4 Hybrid Recommendation Approaches

Hybrid recommendation combines two or more sources of evidence within a single recommendation process. In addition to content-based and collaborative signals, a hybrid system may incorporate popularity, contextual information, business rules and eligibility constraints. The contribution of each component may be fixed or adjusted according to the amount and quality of available evidence. The main purpose of this integration is to improve recommendation robustness when one information source is limited, inconsistent or unavailable (Singh & Dwivedi, 2023).

Several studies have reported benefits from combining content and collaborative evidence under their respective evaluation conditions. For example, Smart Dine-In integrates ingredient-based preference information with collaborative evidence derived from user interactions (Keshav et al., 2023). This structure allows recommendations to consider both stated food preferences and collective behavioural patterns. In another food-recommendation study, Sharma et al. (2023) combined collaborative information with food-attribute profiling to address cold-start limitations and broaden the available recommendation evidence. Taken together, these studies suggest that different information sources can provide complementary support rather than requiring the system to depend on one method alone.

Hybrid evidence can be combined through several strategies, including weighted scoring, sequential filtering and switching between methods according to the available information. For example, a system may first apply ingredient or constraint information to remove unsuitable dishes. Collaborative evidence can then be used to reorder the remaining candidates according to historical interaction patterns. Alternatively, content and collaborative scores may be calculated separately and combined through weighted scoring. The selected strategy should therefore reflect the available data and the intended recommendation behaviour rather than being treated as a universal structure.

Despite these advantages, hybrid approaches introduce additional design trade-offs. Combining multiple signals increases the number of rules, weights and interactions that must be

managed. If one component receives excessive influence, it may dominate the ranking and reduce the contribution of the remaining signals. For instance, overemphasising ingredient similarity may repeatedly favour closely related dishes and limit menu discovery. By contrast, assigning too much weight to collaborative evidence may reintroduce sparsity problems when only a small number of interactions are available (Sharma et al., 2023). Therefore, the relative contribution of each component should be determined according to the evidence available in the current recommendation context.

Overall, the reviewed studies suggest that hybrid recommendation can combine complementary sources of information in food-related systems. Ingredient-based evidence can represent explicit customer preferences, whereas collaborative evidence can reveal relationships derived from previous ordering behaviour. Accordingly, the present study investigates an adaptive and explainable hybrid approach that combines ingredient matching with item-to-item co-order evidence instead of assuming that one fixed weighting is equally suitable for every recommendation request.

2.5 Advanced and Multimodal Recommendation Approaches

Recent studies have explored advanced recommendation models that integrate multiple forms of food and interaction data, including ingredient lists, food images, textual descriptions and user behaviour. Deep-learning and graph-based architecture can model complex relationships among users, recipes, ingredients and other food attributes. Compared with simpler content or collaborative methods, these approaches aim to learn richer representations from several connected sources of information.

For example, Tian et al. (2022) proposed a Hierarchical Graph Attention Network that models structured relationships among users, recipes and ingredients. The authors reported improved recommendation performance on the datasets used in their evaluation, suggesting that graph structures can represent relationships that may not be captured through conventional similarity calculations. In a different multimodal approach, Zhang et al. (2025) introduced CLUSSL, which combines clustering and self-supervised learning to integrate several forms of recipe information. Their study reported strong performance within the evaluated food-

recommendation tasks. Taken together, these studies illustrate how advanced models can incorporate richer information than methods based only on ratings or individual menu attributes.

However, the use of advanced models introduces practical trade-offs. These approaches commonly depend on richer datasets, model training and greater computational resources than lightweight rule-based methods. Moreover, their internal representations may be less straightforward to interpret, making it more difficult to explain how individual ingredients, interactions or contextual factors influenced a recommendation. Although graph-based and multimodal models represent an important direction in food-recommendation research, their data and implementation requirements may not be proportionate to the scale and objectives of the present single-restaurant prototype. Accordingly, this study prioritises a lightweight and explainable hybrid approach that can operate with limited historical data and produce recommendation reasons that can be inspected directly.

2.6 Explainable Recommendation

Explainable recommendation refers to the ability of a recommender system to provide understandable reasons for its suggestions. More specifically, instead of presenting only a ranked list or numerical score, an explainable system may identify the item features, behavioural relationships or contextual factors that influenced a recommendation. Zhang and Chen (2020) noted that such explanations may support transparency, user understanding, trust and system debugging. To remain meaningful, however, the displayed explanation should accurately reflect the evidence and calculations used by the recommendation process.

In food recommendation, explainability is particularly important because customers may wish to know whether a dish was suggested because of ingredient compatibility, dietary preferences, previous ordering behaviour or general popularity. In addition to identifying the factors behind a recommendation, the explanation should avoid overstating the strength of the available evidence. For example, a dish pair that has appeared together in only one historical basket should not be described as having a well-established behavioural relationship.

Drawing on this principle, the present study distinguishes between ranking score and evidence confidence. The ranking score determines the relative order of eligible dishes, whereas

evidence confidence indicates the amount and strength of the information supporting each result. Consequently, a dish may rank highly because it closely matches an explicit ingredient preference even when little historical evidence is available. Separating these concepts allows the system to communicate both why the dish was ranked and how strongly the result is supported. Nevertheless, providing an explanation does not by itself guarantee that customers will understand or trust it, and its usefulness would require evaluation with actual users.

2.7 Key Studies

2.7.1 Key Study 1: Munaji & Emanuel (2021)

Munaji and Emanuel (2021) developed a restaurant recommendation system using user-based collaborative filtering and the Pearson correlation coefficient. The method analysed similarities among users according to their explicit dish ratings and used these relationships to recommend items preferred by similar users. The study demonstrated the potential of user-based collaborative filtering to generate relevant restaurant recommendations when sufficient rating data were available.

However, the method depends strongly on customers providing explicit ratings. In a single-restaurant ordering environment, customers may complete their orders without evaluating individual dishes, resulting in a sparse user–item rating matrix. Under such conditions, user similarities may be based on insufficient evidence, reducing the reliability of the resulting recommendations. The study therefore illustrates both the usefulness of collaborative filtering and its dependence on an adequately populated interaction dataset.

This limitation is relevant to the present study because the developed system does not require explicit dish ratings or permanent customer profiles. Instead, completed orders are treated as baskets, and dishes that occur together are used to derive item-to-item co-order relationships. This retains the collaborative principle of learning from collective behaviour while adapting the source of evidence to data generated naturally through the restaurant-ordering process.

2.7.2 Key Study 2: Shinagam (2025)

Shinagam (2025) compared collaborative filtering and content-based filtering under sparse-data conditions. The content-based approach represented items through recorded attributes and calculated similarities without depending entirely on explicit user ratings. This allowed recommendations to be generated even when the available interaction matrix contained limited information.

The reported findings indicated that content-based information remained useful when collaborative evidence was sparse. Under the evaluated conditions, item attributes provided an alternative source of recommendation evidence when user ratings were insufficient. However, the

results should not be interpreted as proof that ingredient-based methods are universally superior to collaborative filtering or that ingredient similarity directly represents customer taste. Their effectiveness depends on the quality of the recorded attributes, the dataset and the evaluation procedure.

The study is relevant to the present project because it demonstrates the value of item-level information in a limited-data environment. Whereas Shinagam used a vector-based content representation, the present system applies direct ingredient and tag matching. This simpler approach was selected because the menu is small, the scoring process needs to remain deterministic and the recommendation reasons should be easy to inspect. Nevertheless, ingredient matching cannot reveal every relationship between dishes, particularly combinations that customers frequently order together. For this reason, it is used alongside item-to-item co-order evidence rather than as the only recommendation method.

2.7.3 Key Study 3: Keshav et al. (2023)

Keshav et al. (2023) introduced Smart Dine-In, a hybrid recommendation system designed to personalize food discovery. The model integrates ingredient-based preference matching with collaborative filtering, aiming to leverage both explicit user tastes and implicit behavioural patterns.

The study showed that hybrid recommendation systems improve relevance and diversity compared to single-method systems. By combining content-based and collaborative signals, Smart Dine-In achieved better menu exploration and reduced cold-start effects.

Although effective, the integration significantly increases system complexity. The present study draws inspiration from this hybrid principle but seeks a more streamlined implementation. Therefore, we adopt a lightweight architecture that prioritizes simple ingredient list matching and implicit co-ordering patterns. This design is tailored specifically for the practical constraints of a single-restaurant QR ordering system with its core focus on achieving simplicity and efficiency.

2.8 Research Gap

Existing studies have demonstrated that content-based and collaborative recommendation methods can support personalized food selection. Collaborative filtering commonly uses user ratings, purchase histories or similarities among users and items to identify relevant recommendations. For example, Munaji and Emanuel (2021) applied user-based collaborative filtering to restaurant recommendations using explicit ratings, while Sharma et al. (2023) and Singh and Dwivedi (2023) examined the use of collaborative and content-based techniques in food recommendation. These studies show that behavioural data can improve personalization when sufficient user interactions are available.

However, rating-based collaborative filtering may be difficult to apply in a small and single-restaurant environment. Customers may place an order without providing ratings and the number of historical transactions may be significantly lower than on large food delivery platforms. This creates data sparsity and cold-start problems, where new customers and less frequently ordered dishes have insufficient behavioural information. As a result, recommendations that depend mainly on user ratings or extensive interaction histories may not perform consistently in a limited-data setting.

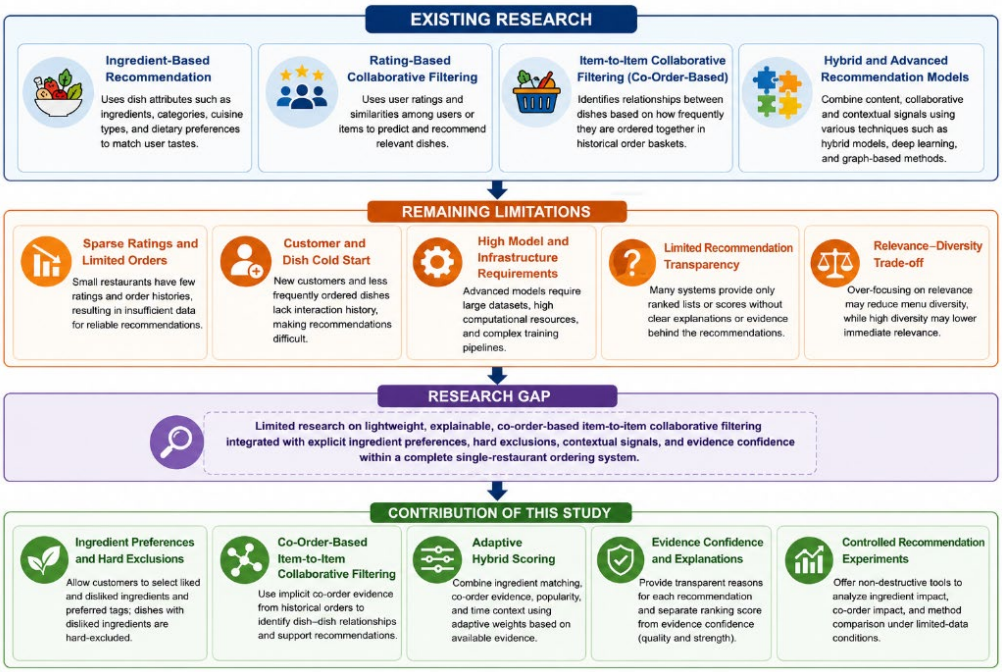
Ingredient-based recommendation can partially address this limitation because menu items can be matched directly with customers stated likes, dislikes and preferred food attributes. Ingredient information is available even when a customer has no previous order history. However, matching ingredients alone cannot identify behavioural relationships between dishes such as items that are frequently purchased together. It may also repeatedly recommend dishes with similar ingredients and provide limited support for meal combination or menu discovery. Therefore, relying only on ingredient similarity may not fully represent customers' actual ordering behaviour.

Hybrid food recommendation systems have been proposed to combine content and behavioural signals. For instance, Keshav et al. (2023) integrated preference information with collaborative recommendation while Singh and Dwivedi (2023) examined the combination of content-based and collaborative techniques. Nevertheless, many existing studies focus mainly on recommendation accuracy or use models that assume richer datasets, explicit user profiles or more complex computational procedures. Advanced approaches such as graph-based and multimodal recommendation may provide strong predictive performance but they generally require larger

datasets, model training and more computational resources (Tian et al., 2022). These requirements may not be practical for a lightweight restaurant prototype.

Accordingly, the research gap addressed by this project is the lack of a lightweight, explainable and practical recommendation approach designed specifically for a limited-data and single-restaurant ordering environment. Few studies have address:

- explicit liked and disliked ingredient preferences;
- hard exclusion of unsuitable dishes;
- implicit item-to-item co-order relationships;
- popularity and contextual fallback signals;
- adaptive weighting based on available evidence;
- separation of ranking score and evidence confidence;
- recommendation explanations;
- diversity and meal-set generation; and
- controlled experiments that isolate ingredients and co-order effects.



This study addresses the identified gap by developing a mobile-first restaurant ordering system that combines ingredient-based matching with collaborative filtering specifically an item-to-item recommendation component. The system uses deterministic and explainable scoring rules instead

of a heavyweight machine-learning model. It also provides controlled experiments for examining how ingredient preferences, simulated co-order evidence and different recommendation methods affect the resulting rankings. The research therefore contributes a practical approach for studying personalized food recommendation under the data and infrastructure limitations of a small restaurant.

Chapter 3: Methodology

3.1 Research Design and Development Framework

This section explains the research approach adopted to develop and evaluate the Personalized Restaurant Ordering System. The methodology combines a **Design Science Research-oriented approach** with **iterative and incremental software prototyping**. The approach was selected because the project involves designing a functional computing artefact, integrating it into a realistic restaurant-ordering workflow and evaluating both its system functions and recommendation behaviour.

3.1.1 Overview of the System Development Approach

Design Science Research is suitable for applied software-engineering research where a technological artefact is created to address an identified practical problem. A design-science perspective connects the development of the artefact with its practical relevance, technical contribution and evaluation (Baskerville et al., 2018).

The main artefact developed in this study is a mobile-first web application that provides:

System area	Main purpose
Customer ordering system	Supports menu browsing, preference selection, recommendations, cart management, checkout and order tracking
Administrator system	Supports order management, dish management, operational insights and recommendation testing
Recommendation engine	Generates explainable dish recommendations using ingredients, co-order, popularity and time-context signals
Research experiment tools	Examine ingredient impact, co-order impact and differences between recommendation methods

The implemented system addresses three practical conditions associated with a small restaurant:

1. New customers may have no previous ordering history.

2. Historical co-order evidence may be limited; and
3. Customers and administrators need understandable reasons for recommendations.

The system lastly uses transparent deterministic rules rather than an opaque trained model. Its recommendation engine combines explicit ingredient preferences, co-order-based item-to-item collaborative filtering, dish popularity and time context. Besides, controlled experiments are kept separate from operational recommendation behaviour so that simulated scenarios do not change the real historical order data.

Research-design element	Description
Research orientation	Design Science Research-oriented development
Development strategy	Iterative and incremental prototyping
Research problem	Personalised restaurant recommendation under limited-data conditions
Primary artefact	Personalised Restaurant Ordering System
Application type	Mobile-first browser-based web application
Main users	Customers and restaurant administrators
Recommendation approach	Ingredient-based matching and co-order-based item-to-item collaborative filtering
Additional signals	Popularity and time context
Constraint mechanism	Hard exclusion of dishes containing disliked ingredients
Evaluation approach	System testing and controlled recommendation experiments
Main evaluation outputs	Ranking changes, Hit@K, hidden-dish rank, preference match, restriction violations and association evidence

Table 3.1: Research Design Summary

The system-development process was not strictly linear. Each major system component passed through repeated planning, implementation, testing and refinement. In addition, software-prototyping research identifies exploration, incremental development, quality improvement, validation and testing as important purposes of prototyping. Therefore, prototypes can support both the gradual clarification of requirements and the evaluation of proposed solutions (Bjarnason et al., 2023).

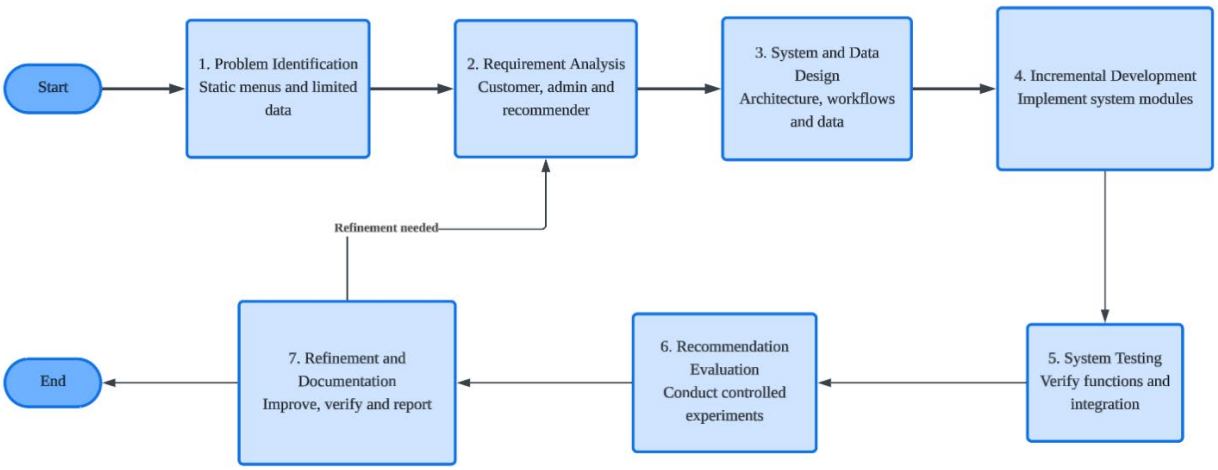


Figure 3.1: Overall Research and Development Process

The feedback arrow shows that findings from testing and evaluation were used to refine earlier requirements and implementation decisions. For example, customer and administrator interfaces were separated, completed orders were connected to historical recommendation evidence and controlled experiments were isolated from the operational dataset.

3.1.2 Justification of the Selected Development Methodology

The ***Design Science Research-oriented methodology*** was selected because the objective of the study was not limited to analysing existing recommendation techniques. The project required the design, implementation and evaluation of a complete software artefact capable of demonstrating how recommendation methods operate within a restaurant-ordering environment.

The methodology was supported by iterative and incremental prototyping because several project requirements became clearer during development. The customer ordering flow was first implemented and tested before being connected to administrator order management. The recommendation engine was subsequently introduced after the menu, cart, checkout and order-tracking functions were operational. Controlled recommendation experiments were added after the production recommendation pipeline had been established.

Project characteristic	Methodological response	Justification
A working software system was required	Design Science Research orientation	The study produces and evaluates a purposeful computing artefact
Requirements developed progressively	Iterative prototyping	Requirements could be clarified through working versions of the system
The application contains several functional areas	Incremental development	Customer, administrator, persistence and recommendation modules could be developed separately
The project uses limited historical data	Controlled experimentation	Recommendation behaviour could be examined without requiring a large live dataset
The project was developed by one student	Lightweight development process	A formal team-based framework would not accurately represent the project
Recommendations must be understandable	Explainable deterministic design	Component scores, evidence and exclusion reasons can be inspected
Real order data must remain protected	Non-destructive experiments	Temporary simulated evidence is kept separate from operational history
The system must work before recommendation evaluation	Staged implementation	Functional ordering components were verified before controlled experiments

Table 3.2.1: Justification of the Selected Methodology

Comparison with Alternative Approaches

Approach	Suitability	Reason
Waterfall	Limited suitability	Requirements and interface behaviour changed during implementation, making a fully fixed sequential process restrictive
Formal Scrum	Not selected	The project did not involve a Scrum team, product owner or formal sprint ceremonies
Machine-learning experimentation only	Not sufficient	The project required a complete restaurant ordering application, not only an isolated recommendation model
Design Science with iterative prototyping	Selected	Supports artefact development, practical problem-solving, evaluation and progressive refinement

Table 3.2.2: Comparison with Selected Methodology

The project is therefore described as using a **Design Science Research-oriented methodology** rather than claiming strict compliance with a formal Design Science Research Methodology framework. Similarly, the development process is described as iterative and incremental rather than as formal Scrum.

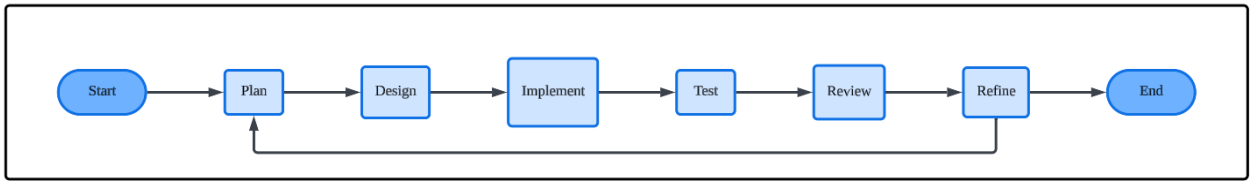


Figure 3.2: Iterative Development Cycle

This iterative cycle was applied at different levels of the project. For example:

Module	Initial implementation	Subsequent refinement
Menu interface	Display menu items and categories	Added search locator, dish details and responsive behaviour

Cart	Add dishes and display item count	Added quantities, removal, subtotal calculation and checkout
Order management	Create customer orders	Added administrator status updates and customer-side tracking
Recommendation engine	Ingredient and co-order scoring	Added adaptive weights, confidence, diversity and explanations
Evaluation tools	View recommendation output	Added controlled ingredients, co-order and method-comparison experiments

3.1.3 System Development Phases

The project was organised into eight development phases. Although the phases are presented sequentially, some activities overlapped, particularly testing, interface refinement and documentation.

Phase
1. Problem identification and planning
2. Requirement identification
3. Reference review and technology selection
4. Architecture and data design
5. Customer-side development
6. Administrator-side development
7. Recommendation and experiment development
8. Testing, refinement and reporting

Table 3.3: System Development Phases

The ***first phase*** involved reviewing the original project proposal and confirming the project scope as a single-restaurant ordering and recommendation system. The problem was examined from both the customer and restaurant operator perspectives. The main concerns identified included static digital menus, limited historical order data, customer cold start and the need for understandable recommendation results.

During the ***second phase***, the project requirements were identified and grouped into four areas: customer functions, administrator functions, recommendation functions and research experiment functions. This separation helped distinguish the operational restaurant system from the recommendation engine and its evaluation tools.

The ***third phase*** involved reviewing an existing restaurant ordering system to understand common customer and administrator workflows. Based on the project requirements and practical constraints, the project was changed from an earlier desktop concept to a mobile-first web application. Rust was retained as the main programming language while Axum and Tokio were selected for web routing and asynchronous server operations.

The ***fourth phase*** focused on the system architecture and data design. The application was divided into modules for web routing, request handling, application state, recommendation logic, search, preference processing and persistence. Dish records and historical order baskets were also defined before the recommendation logic was implemented. CSV and JSONL files were selected for prototype data persistence instead of an external database.

The ***fifth phase*** focused on customer-side development. The menu interface, dish search, category navigation, preference selection, personalized recommendations, meal-set generation, cart management, checkout and order-status tracking were developed incrementally. The interface was designed for mobile access so that customers could use their own phones when ordering.

The ***sixth phase*** focused on administrator-side development. A separate administrator interface was implemented for authentication, order viewing, order-status updates, dish management, recommendation insights and research tools. Completed customer orders were also connected to the historical order data so that confirmed transactions could contribute to future dish-popularity and co-order evidence.

The ***seventh phase*** involved developing the recommendation engine and controlled experiment tools. The recommendation engine combined ingredient-based matching, co-order-based item-to-item collaborative filtering, popularity and time context. Disliked ingredients were handled as hard exclusions, while adaptive weighting determined the contribution of each available recommendation signal. Controlled experiments were then developed to examine ingredient impact, co-order impact and differences among ingredient-only, co-order-only and hybrid recommendation methods.

The ***final phase*** involved system testing, recommendation evaluation, refinement, and documentation. Customer and administrator workflows were tested individually and as complete end-to-end processes. Security, responsive layout, data persistence, deterministic recommendation behaviour, hard exclusions, adaptive weights and experiment isolation were also verified. Findings from these tests were used to correct implementation issues and prepare the final report.

Overall, the development process progressed from problem identification and requirement analysis to artefact construction, testing and controlled evaluation. This ensured that the recommendation methods were studied within a functioning restaurant ordering system rather than as isolated algorithms.

3.2 Requirement Analysis

3.2.1 Functional Requirements

Functional requirements define the operations that the system must perform. They were organised into four groups:

1. customer functions;
2. administrator functions;
3. recommendation functions; and
4. controlled research experiment functions.

The requirement identifiers established here can later be reused in Chapter 4's requirement-fulfilment matrix.

1. Customer Functional Requirements

ID	Category	Functional requirement
CFR-01	Customer	The system shall allow a customer to create a temporary restaurant session using a name, Malaysian telephone number, and table identifier.
CFR-02	Customer	The system shall allow customers to browse the complete restaurant menu by category.
CFR-03	Customer	The system shall provide a search locator using dish names, categories, ingredients, tags, and dish identifiers.
CFR-04	Customer	The system shall display dish information, including the dish name, category, ingredients, tags, image, and prototype price.
CFR-05	Customer	The system shall allow customers to select liked ingredients, disliked ingredients, preferred tags, and context dishes.
CFR-06	Customer	The system shall generate personalised dish recommendations based on the customer's current preference and context inputs.
CFR-07	Customer	The system shall present plain-language reasons and evidence indicators for recommended dishes.
CFR-08	Customer	The system shall allow customers to select Familiar, Balanced, or Discover recommendation-diversity modes.

CFR-09	Customer	The system shall generate meal-set suggestions based on budget, party size, category requirements, preferences, and restrictions.
CFR-10	Customer	The system shall allow customers to add dishes to a cart, change quantities, remove items, and view the subtotal.
CFR-11	Customer	The system shall allow customers to submit the cart as a restaurant order.
CFR-12	Customer	The system shall allow customers to view their orders and automatically track order-status changes.

2. Administrator Functional Requirements

ID	Category	Functional requirement
AFR-01	Administrator	The system should provide a separately scoped administrator login.
AFR-02	Administrator	The system shall prevent unauthenticated users from accessing protected administrator pages and APIs.
AFR-03	Administrator	The system shall display dashboard summaries, dish popularity, and named co-order relationships.
AFR-04	Administrator	The system shall allow administrators to view live and historical orders.
AFR-05	Administrator	The system shall allow administrators to change an order's status among Pending, Preparing, Ready, Completed, and Cancelled.
AFR-06	Administrator	The system shall allow administrators to create, view, edit, delete, and change the availability of dishes.
AFR-07	Administrator	The system should protect stable dish identifiers when existing dishes are edited.
AFR-08	Administrator	The system should provide dish-image previews and menu CSV export.

3. Recommendation Functional Requirements

ID	Category	Functional requirement
RFR-01	Recommendation	The system shall calculate ingredient-based relevance using liked ingredients and preferred dish tags.
RFR-02	Recommendation	The system shall exclude unavailable dishes, already-selected dishes, and dishes containing disliked ingredients before ranking.
RFR-03	Recommendation	The system shall perform co-order-based item-to-item collaborative filtering using historical order baskets.

RFR-04	Recommendation	The system shall calculate popularity and time-context scores as supporting recommendation signals.
RFR-05	Recommendation	The system shall adjust recommendation-component weights according to the content and co-order evidence available.
RFR-06	Recommendation	The system shall ensure that all component weights sum to 1.0.
RFR-07	Recommendation	The system shall separate recommendation ranking scores from evidence-confidence indicators.
RFR-08	Recommendation	The system shall perform diversity reranking without reintroducing ineligible dishes.
RFR-09	Recommendation	The system shall provide explanations identifying the principal signals affecting each recommendation.
RFR-10	Recommendation	The system shall use completed customer orders to update later popularity and co-order evidence.

4. Experiment Functional Requirements

ID	Category	Functional requirement
EFR-01	Experiment	The system shall provide an Ingredient Preference Impact experiment comparing neutral and preference-shaped rankings.
EFR-02	Experiment	The system shall provide a Co-Order History Impact experiment using temporary simulated co-order baskets.
EFR-03	Experiment	The system shall compare ingredient-only, co-order-only, and fixed hybrid recommendation methods.
EFR-04	Experiment	The system shall support leave-one-item-out testing using a hidden dish from a historical basket.
EFR-05	Experiment	The system shall report metrics such as rank, Hit@K, preference match, restriction violations, pair count, support, confidence, and lift.
EFR-06	Experiment	The system shall provide counterfactual analysis showing how temporary changes affect rankings and weights.
EFR-07	Experiment	Controlled experiments shall not modify real historical orders or operational learning records.

The customer and administrator's requirements show the final application scope documented in the repository. The customer workflow covers temporary registration, menu browsing, preferences, recommendations, meal sets, cart, checkout and tracking while the administrator interface covers orders, dishes, insights, and recommendation tools.

The recommendation requirements also distinguish **production behaviour** from **controlled evaluation**. Production recommendations use adaptive weights according to available evidence whereas controlled experiments use fixed method definitions so that comparisons remain repeatable.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational conditions expected from the prototype. These requirements were organized around usability, responsiveness, reliability, security, maintainability, explainability, data integrity and repeatability.

Non-Functional Requirements

ID	Quality attribute	Requirement and verification criterion
NFR-01	Usability	The customer interface shall present clear menu navigation, preference controls, recommendation explanations, cart feedback, and order-status information. Verification is performed through end-to-end workflow inspection.
NFR-02	Mobile responsiveness	The customer interface shall remain usable on mobile, tablet, and desktop viewports without overlapping controls or inaccessible content.
NFR-03	Compatibility	The application shall operate through a modern web browser without requiring a dedicated customer application or restaurant tablet.
NFR-04	Determinism	Identical recommendation inputs and historical data shall produce identical recommendation results.
NFR-05	Reliability	Customer ordering and administrator-management workflows shall complete without invalid state transitions or data loss during the local prototype demonstration.
NFR-06	Security	Administrator pages and mutation APIs shall require an authenticated administrator session that is separate from customer sessions.
NFR-07	Secure configuration	Administrator credentials shall be provided through environment configuration, with no built-in default credentials.
NFR-08	Data integrity	The data loader shall reject missing required columns, duplicate dish identifiers, invalid timestamps, and references to unknown dishes.
NFR-09	Persistence safety	Completed orders shall be added to historical data only once, while incomplete and cancelled orders shall not be used as confirmed recommendation evidence.
NFR-10	Experiment isolation	Temporary experiment inputs shall remain in memory and shall not alter operational CSV, JSONL, or historical-order records.
NFR-11	Explainability	Each recommendation shall provide an understandable reason, component information, or evidence indicator sufficient to inspect why the dish was ranked.

NFR-12	Maintainability	Routing, request handling, recommendation logic, search, preference processing, persistence, and presentation shall remain separated into dedicated modules.
NFR-13	Performance	Menu browsing, order actions, and recommendation requests shall complete without noticeable delay under the prototype dataset and local demonstration environment.
NFR-14	Repeatability	Controlled experiments shall use fixed procedures and method definitions so that the same scenario can be reproduced.
NFR-15	Restriction compliance	A popularity or co-order score shall never override the hard exclusion of a disliked ingredient.

The prototype uses a modular Rust architecture in which routes and handlers manage HTTP communication, recommendation modules own the scoring formulas and persistence modules manage file safety. This separation supports maintainability and reduces coupling between the customer interface, administrator interface, recommendation engine, and storage layer.

Moreover, the system also validates required dish and historical-order fields before those records are used by the recommender. Duplicate dish identifiers, invalid timestamps, and unknown dish references could otherwise produce incorrect popularity and co-order results.

3.2.3 User Roles and Access Requirements

The prototype defines two human user roles and one internal system role.

1. **Customer**

   A customer creates a temporary restaurant session and can:

   - browse and search the menu
   - select preferences and context dishes
   - request personalized recommendations
   - generate meal sets
   - manage the cart
   - check out
   - view orders associated with the current customer session.

   Customers cannot access administrator pages, modify dishes, view other customers' orders or change order statuses.

2. **Administrator**

   An administrator authenticates through separately scoped credentials and can:

   - view the operational dashboard
   - view live and historical orders
   - update order statuses
   - manage dishes and availability
   - view popularity and co-order insights
   - use production and controlled recommendation tools.

   The administrator does not use the customer-session cookie as authorization for protected administrator actions. Administrator credentials are configured through environment variables and missing credentials produce a configuration error instead of an insecure default login.

3. **System**

   The system acts as an internal automated role responsible for:

   - validating requests and datasets;
   - maintaining application state;

- generating recommendations;
- applying hard exclusions;
- calculating evidence and scores;
- synchronising order-status information;
- persisting completed orders; and
- protecting real order history during controlled experiments.

Function	Customer	Administrator	System
Browse and search menu	✓	—	Provides data
Select preferences	✓	Test tools only	Processes inputs
Receive recommendations	✓	Tester access	Generates results
Manage cart and checkout	✓	—	Validates and creates order
View own order status	✓	—	Synchronises status
View all live and historical orders	—	✓	Retrieves records
Change order status	—	✓	Validates transition
Manage dishes and availability	—	✓	Validates and persists changes
View recommendation insights	—	✓	Calculates evidence
Run controlled experiments	—	✓	Isolates temporary data
Modify administrator credentials	—	Environment configuration only	Loads configuration

Table 3.4: Role and Access Matrix

This role separation protects operational functions and supports the project's requirement that the customer and administrator interfaces remain clearly distinct. The implementation provides separately scoped administrator access and customer-specific order tracking.

3.2.4 System Constraints and Assumptions

The project was developed as an academic prototype rather than a production restaurant platform. Its requirements were therefore defined within specific technical and research constraints.

System Constraints

1. **Single restaurants scope**
   The application is designed for one restaurant, one menu and one local application process. Multi-restaurant tenancy is outside the current scope.
2. **Limited historical data**
   The recommendation engine operates on a relatively small historical-order dataset. Co-order evidence may therefore be sparse, especially for less frequently ordered dishes.
3. **Local persistence**
   CSV and JSONL files are used for prototype persistence. This approach is suitable for controlled demonstration but is not designed for high-volume concurrent production writes.
4. **In-memory operational state**
   Customer sessions, live carts and uncompleted orders are maintained in application memory and may not survive an application restart.
5. **Rule-based parameters**
   Time-context rules, adaptive-weight thresholds, evidence categories and diversity settings are deterministic prototype heuristics rather than statistically learned parameters.
6. **Limited administrator account management**
   Administrator access is configured through environment variables. Password hashing, multiple staff roles, password recovery and account-management interfaces are not implemented.

7. **No certified dietary guarantee**
   Ingredient dislikes are treated as recommendation constraints, but the system does not provide certified allergen, nutrition or medical advice.

8. **Excluded production integrations**
   Online payment, kitchen printers, delivery platforms, real-time stock management and production cloud deployment are outside the project scope.

9. **Interpretation of confidence**
   Evidence confidence indicates the strength of available supporting information. It is not the probability that a customer will like or purchase a dish.

As these limitations are declared in the current study so that the prototype is not represented as a production-ready or statistically predictive system.

System Assumptions

The methodology assumes that:

1. Customers use a modern browser on a phone, tablet, or computer.
2. The restaurant provides valid menu and ingredient information.
3. Dish identifiers remain stable across menu changes and historical orders.
4. A completed order represents valid behavioural evidence.
5. Cancelled or incomplete orders should not influence the historical recommender.
6. Historical order baskets provide sufficient evidence for demonstrating co-order behaviour even though they may not represent a wider restaurant population.
7. The administrator configures the required credentials before starting the application.
8. The application is demonstrated in a controlled local or local network environment.

Theis final set of requirements provides the foundation for the architecture and system design presented in ***Section 3.3***. Their fulfilment and associated testing evidence will be evaluated in ***Chapter 4***.

3.3 System Design

This section describes how the system was structured before and during implementation. It focuses on the relationships between the customer interface, administrator interface, web server, recommendation engine and local persistence.

3.3.1 System Architecture

The system follows a layered web-application architecture. Customers and administrators interact with different browser interfaces but both interfaces communicate with the same Rust application. Axum handles routing and HTTP requests while shared application state coordinates the menu, sessions, orders, recommendation services and persistence operations.

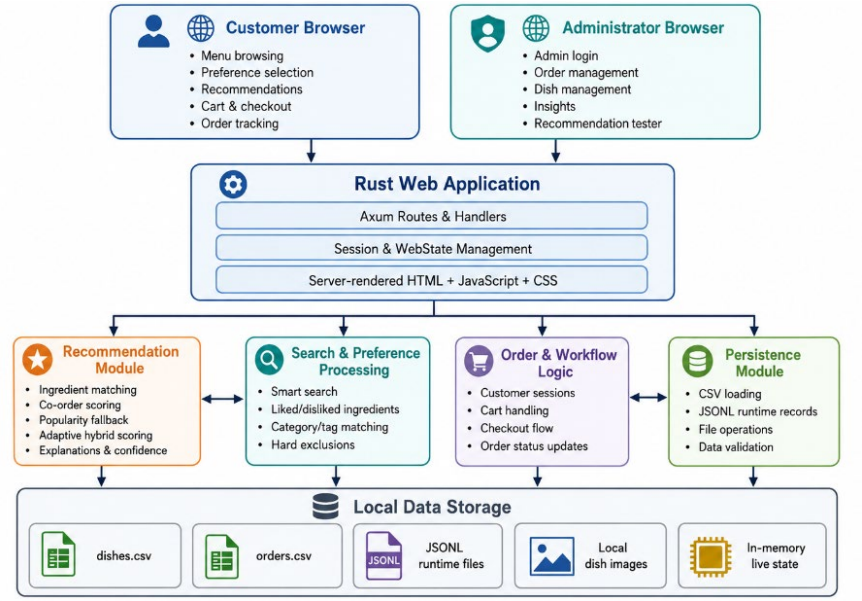


Figure 3.3: High-Level System Architecture

The architecture separates concerns as follows:

- The **presentation layer** displays the customer and administrator interfaces
- The **web layer** receives requests and returns pages or JSON responses
- The **application layer** coordinates business workflows

- The **recommendation layer** performs filtering, scoring, ranking, and explanation generation
- The **data access layer** manages CSV, JSONL, and local image files.

This separation reduces direct dependency between the user interface and data files. For example, the browser does not read historical orders directly. Instead, a request is handled by the application which retrieves the required data and passes it to the recommendation or order-management service.

3.3.2 Use Case Diagram

The system has two main actors:

1. **Customer**
2. **Administrator**

The customer uses the system to browse the menu, provide preferences, obtain recommendations, place orders and track order progress. The administrator manages operational activities and accesses recommendation-analysis tools.

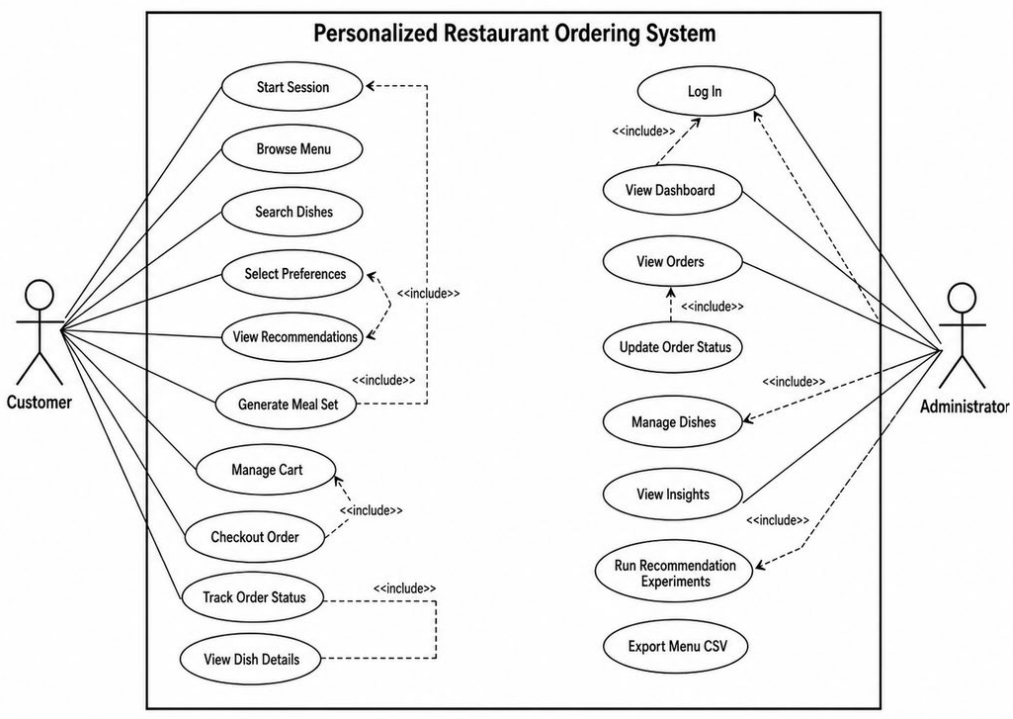


Figure 3.4: System Use Case Diagram

Several use cases depend on earlier actions. For example, a customer must start a valid session before checking out or viewing session-specific orders. Similarly, the administrator must authenticate before accessing protected management pages or experiment tools.

The recommendation use case is connected to preference selection but is not limited to explicit preferences. The system can also use selected dishes, historical co-order evidence, popularity and time context when producing recommendations.

3.3.3 Customer and Administrator Activity Diagram

A combined activity diagram was used to show how the customer, administrator, and system interact during the ordering process. This avoids presenting separate diagrams that repeat the same order lifecycle.

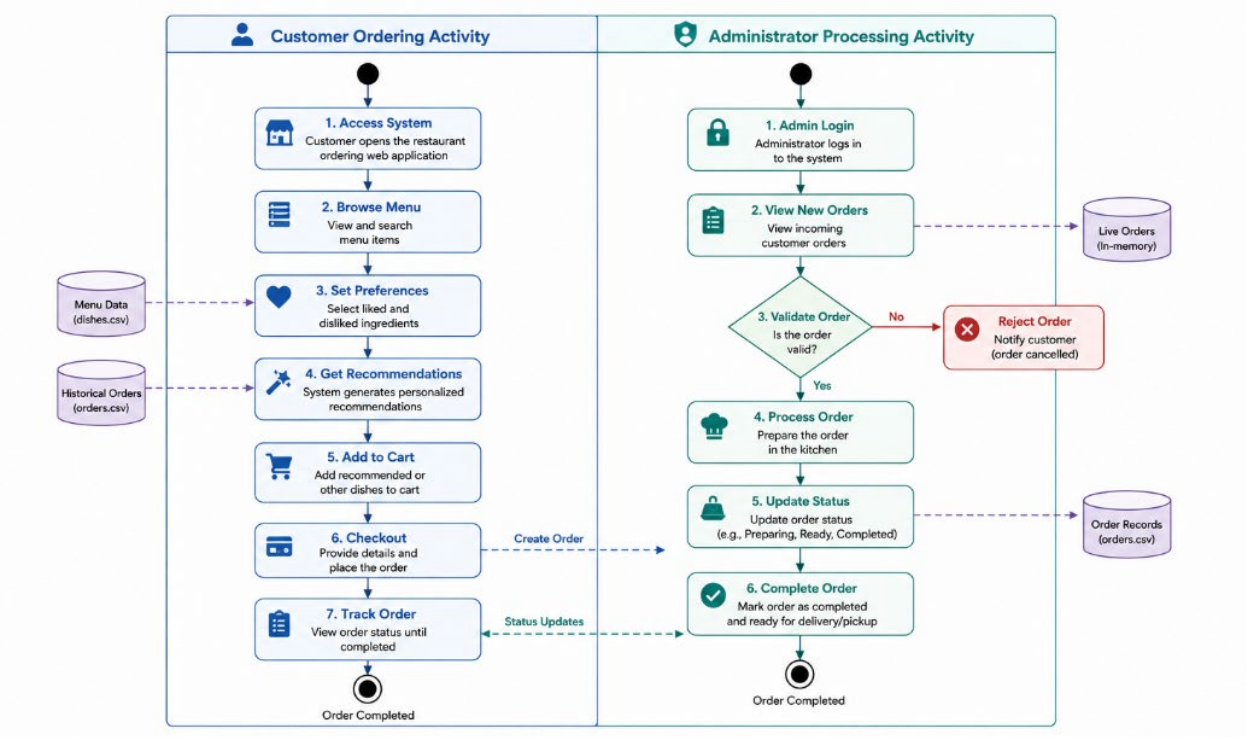


Figure 3.5: Customer Ordering and Administrator Processing Activity

The status sequence is designed around the typical progression:

$$Pending \rightarrow Preparing \rightarrow Ready \rightarrow Completed$$

An order may be marked as ***Cancelled.*** Only confirmed completed orders should contribute to later popularity and co-order evidence. Therefore, keeping live and historical orders separate helps to prevent incomplete orders from influencing the recommendation engine.

3.3.4 Conceptual Data Model

The project does not use a relational database server. Therefore, a traditional database entity-relationship diagram would inaccurately describe the implementation. Instead, a **conceptual data model** is used to show the main data objects and their relationships.

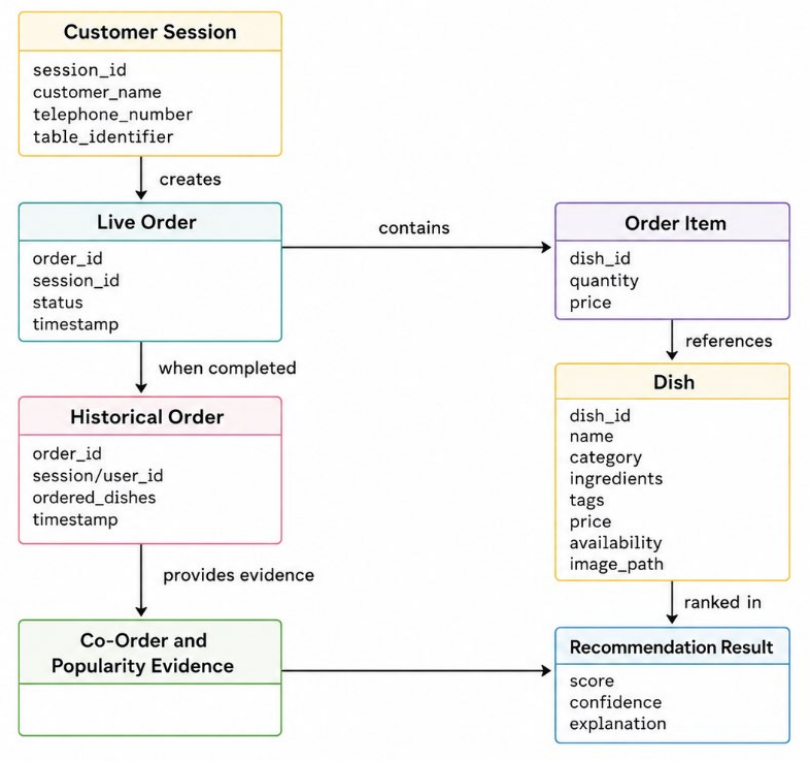


Figure 3.6: Conceptual Data Relationship Diagram

A dish identifier acts as the connection between menu data, order items, historical baskets and recommendation evidence. Stable identifiers are therefore very important. Not only that, changing an identifier after historical orders have been recorded could break the relationship between the menu and earlier order baskets.

3.3.5 Data Access Object Design

Local file storage was selected to keep the prototype lightweight and easy to demonstrate. The design separates stable menu data, historical recommendation evidence and runtime operational records.

Data source	Main content	Purpose
Dish CSV	Dish IDs, names, ingredients, categories, tags, prices, availability, and image paths	Stores the active menu used by the customer interface and recommender
Historical-order CSV	Order IDs, session or user IDs, ordered dish baskets, and timestamps	Provides popularity and co-order evidence
Runtime JSONL records	Detailed operational orders and recommendation-learning events	Stores records that do not fit the simpler historical basket format
Local image directory	Dish photographs and image assets	Supports menu and administrator interfaces
In-memory application state	Sessions, live orders, loaded menu data, and active services	Supports current application operation

Table 3.5: Data Access Object Design

The data access object design includes validation before information is used by the system. Required fields must be present, dish identifiers must be unique, timestamps must be valid and historical orders must not reference unknown dishes. These checks are important because invalid menus or historical data could produce incorrect co-order relationships and recommendation results. The current project structure also separates web, recommender, storage, search, preference, model and static-resource components.

CSV was selected because the menu and historical order baskets are small and easy to inspect manually. JSONL supports records that may contain more detailed or variable fields. This, this approach is appropriate for the prototype but is recognized as a constraint for future high-volume or multi-user production use.

3.3.6 Recommendation Engine Design

The recommendation engine was designed as a deterministic decision-support model rather than a statistically trained model. It combines several understandable signals while ensuring that restrictions are applied before ranking.

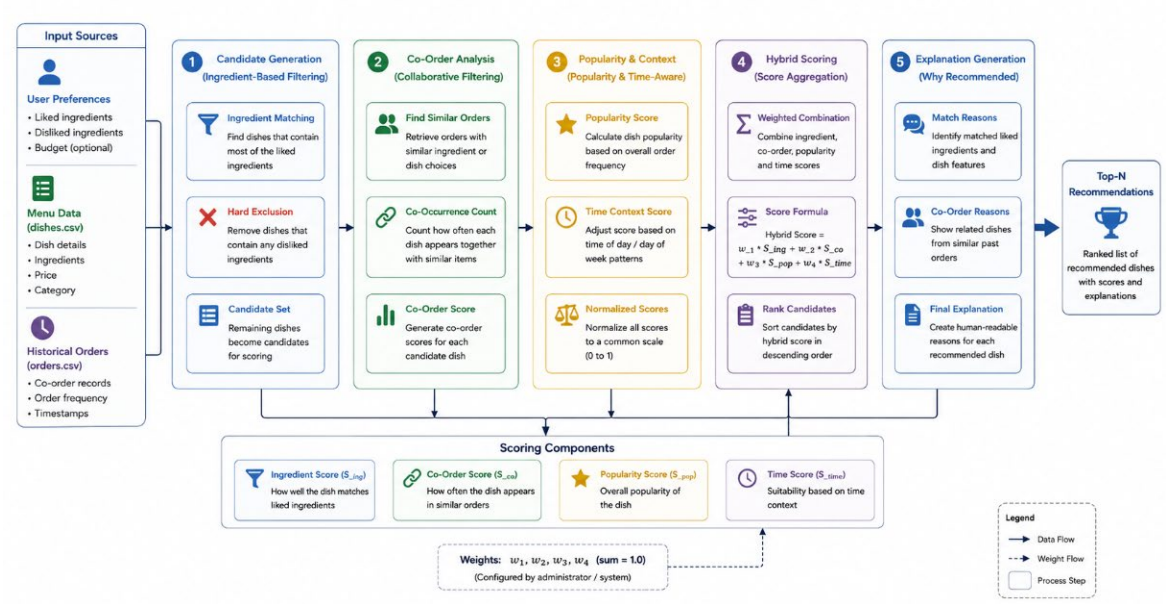


Figure 3.7: Recommendation Processing Pipeline

Eligibility filtering: it occurs before score calculation. A dish is not eligible when it is:

- unavailable;
- already selected by the customer; or
- identified as containing a disliked ingredient.

This ordering prevents a high popularity or collaborative score from overriding an explicit restriction. The hard-exclusion-first approach was implemented as a core design decision in the recommendation system.

Ingredient-based content score: The content component compares the customer's explicit preferences with the characteristics of each eligible dish. Liked ingredients and preferred tags contribute positively to the score. Whereas disliked ingredients are not treated as a negative soft score because they are handled earlier as hard exclusions.

The content score may be represented generally as:

$$C_i = f(L_i, T_i)$$

where:

- C_i is the content score for dish i;
- L_i represents liked ingredient matches; and
- T_i represents preferred-tag or category matches.

Co-order-based item-to-item collaborative score: Historical orders are treated as baskets. When two dishes occur within the same basket, the relationship between them is recorded as co-order evidence. With that, a candidate dish will receive collaborative support when it has previously appeared with one or more dishes currently selected by the customer. This method is described as *co-order-based item-to-item collaborative filtering* because it derives relationships between dishes using implicit ordering behaviour rather than user ratings.

Supporting scores: Popularity provides a fallback signal when ingredient or co-order information is weak. Moreover, the time context adds simple business rules based on the current ordering period. These supporting signals are not intended to replace personalization but help the system produce a deterministic result when stronger evidence is unavailable.

Adaptive hybrid score: The general hybrid score for candidate dish i is:

$$S_i = w_c C_i + w_{co} CO_i + W_p P_i + w_t T_i$$

where:

- S_i is the base recommendation score
- C_i is the ingredient and tag content score
- CO_i is the co-order collaborative score
- P_i is the popularity score
- T_i is the time-context score
- w_c, w_{co}, w_p, w_t are the component weights

The weights satisfy:

$$w_c + w_{co} + w_p + w_t = 1$$

The design uses adaptive weights rather than applying one fixed combination to every production request. For example, when a customer provides explicit preferences but no selected dishes, the content component can receive more influence. When selected dishes have stronger historical associations, the collaborative component can receive greater influence.

Controlled method-comparison experiments are different: they use fixed definitions so that ingredient-only, co-order-only and hybrid outputs can be compared under equivalent conditions.

Diversity reranking: After the base score is calculated, the eligible list may be reranked according to the selected diversity mode:

- **Familiar** prioritizes closer alignment with known preferences and common choices;
- **Balanced** combines relevance with moderate variation; and
- **Discover** gives more opportunity to different or less familiar eligible dishes.

Diversity reranking occurs after eligibility filtering which means it cannot restore an unavailable or restricted dish.

3.3.7 Recommendation Explainability and Evidence Confidence Design

The recommendation for output was designed to present more than a final numerical score. Each result may contain a plain-language explanation, component information and an indication of supporting evidence.

Output element	Meaning
Ranking score	Determines the relative ordering of eligible candidate dishes
Component scores	Show the contribution of content, co-order, popularity, and time signals
Evidence confidence	Indicates the amount and strength of evidence supporting the result
Explanation	Describes the main reasons why the dish was recommended
Exclusion reason	Explains why an unavailable, selected or disliked-ingredient dish was removed

Table 3.6: Recommendation Output Elements

Ranking score and confidence are deliberately separated. A dish may rank highly because it closely matches an explicit preference even when historical evidence is limited. Conversely, a strong co-order relationship may have higher behavioural evidence but still not be eligible if the dish violates a restriction.

3.3.8 Security and Session Design

Customer and administrator access were designed as separate session scopes. A customer begins by entering basic restaurant-session information, after which the system associates the customer's checkout and order-tracking activities with that session. The customer should only be able to view orders linked to the current session.

Administrator access is protected separately. The design requires authenticated administrator access before protected pages or mutation operations can be performed. Administrator credentials are loaded through environment configuration rather than embedded as insecure defaults.

The main security design decisions were:

- separation of customer and administrator session cookies;
- authentication checks before protected administrator operations;
- authorization checks before viewing session-specific orders;
- validation of submitted dish and order identifiers;
- separation of controlled experiment data from operational history; and
- configuration-based administrator credentials.

The prototype does not claim production-level identity management. Those functions such as password recovery, multiple administrator roles, account registration and external identity providers remain outside the current scope.

3.3.9 User Interface and User Experience Design

The user interface was designed with a mobile-first approach because restaurant customers are expected to access the menu through their own phones. The customer interface prioritizes the ordering flow while the administrator interface prioritizes operational visibility and management.

The customer interface was designed around the following principles:

- menu content should be readable on a small screen
- important actions should remain easy to reach
- customers should be able to search without losing the full menu context
- dish details should be viewable without leaving the menu
- cart changes should produce immediate visual feedback
- recommendations should be presented alongside understandable reasons
- order status should refresh without requiring repeated manual page reloads.

The ***customer*** menu includes category navigation, a search locator, recommendation sections, dish cards, detail views, preference controls, cart feedback, and order tracking. The implemented search design locates and focuses on a matching dish rather than permanently hiding the remainder of the menu.

The ***administrator*** interface was designed separately to avoid combining customer-facing functions with operational controls. Orders, dishes, insights and recommendation research tools

are organised into dedicated areas so that the administrator does not need to navigate through the customer ordering interface.

3.4 System Development

While Section 3.3 described the system architecture and design decisions, this section explains how those designs were translated into the implemented application. Development was carried out incrementally. Initially, the basic menu and ordering workflow was implemented. Subsequently, administrator management, recommendation functions, research experiments and completed-order learning were connected to the same application. The final implementation does not require an external database server, frontend framework, large language model or machine-learning framework.

3.4.1 Technology Stack and Development Environment

The technologies were selected according to the requirements of a lightweight, maintainable and mobile-accessible prototype. Rust was used for the server-side implementation whereas standard browser technologies were used for presentation and interaction.

Area	Technology	Purpose
Programming language	Rust 2024 edition	Implements web services, application logic, recommendation algorithms, and persistence
Minimum Rust version	Rust 1.85	Provides compatibility with the selected Rust edition and dependencies
Web framework	Axum 0.7	Defines routes, request handlers, responses, and middleware integration
Asynchronous runtime	Tokio	Supports asynchronous networking and server execution
Data serialisation	Serde and serde_json	Converts Rust structures to and from JSON
CSV processing	csv crate	Loads menu and historical-order records
Date and time	Chrono	Processes timestamps and local time context
Static-file service	tower-http	Serves CSS, JavaScript, and image resources

Front end	Server-rendered HTML	Produces customer and administrator pages
Browser interaction	Plain JavaScript	Provides search, preferences, cart, recommendation refresh, and status updates
Styling	Mobile-first CSS	Supports phone, tablet, and desktop layouts
Persistence	CSV and JSONL files	Stores menu, historical-order, and runtime information
Build and dependency management	Cargo	Builds, runs, tests, and manages Rust dependencies

Table 3.6: Recommendation Output Elements

The project package is configured for Rust 2024 with Rust 1.85 as the minimum version. Its main dependencies include Axum, Tokio, Serde, serde_json, the CSV crate, Chrono and tower-http. Rust was selected because it supports structured modular development, type safety and predictable performance. Meanwhile, Axum provided the HTTP routing and request-handling layer whereas Tokio supported asynchronous server operations. Plain HTML, JavaScript and CSS were retained on the browser side to avoid introducing an additional frontend framework.

3.4.2 Front-End Implementation

The front end was implemented using server-rendered HTML, JavaScript and responsive CSS. The initial page content is generated by the Rust server whereas JavaScript handles interactive actions after the page is loaded.

Two separate interfaces were implemented:

- the customer ordering interface; and
- the administrator management interface.

Customer interface

The customer interface was developed around the main restaurant-ordering journey. It provides:

- temporary customer-session registration;

- category-based menu browsing;
- dish search;
- preference selection;
- personalised recommendations;
- meal-set generation;
- cart and quantity management;
- checkout; and
- order-status tracking.

The menu was designed for mobile use because customers are expected to access the application through their own phones. Therefore, menu cards, preference controls, recommendation panels, cart controls and navigation elements were arranged to remain readable and accessible on smaller screens.

JavaScript was used to process browser-side interactions. For example, when a customer changes a liked or disliked ingredient, the browser sends the updated preferences to the server and retrieves a new recommendation list. Similarly, when a dish is added to the cart, the cart count and subtotal are updated immediately.

The cart stores dish identifiers and quantities in browser storage. However, the server still validates the submitted items during checkout. Consequently, an unavailable or unknown dish cannot be accepted merely because it is present in browser storage.

The search function was implemented as a menu locator rather than a permanent filter. When a matching dish is selected, the interface directs the customer to the relevant menu card while preserving the full menu. This allows customers to locate a dish without losing access to the other available choices.

Administrator interface

In contrast, the administrator interface was designed for operational management. It provides access to:

- dashboard summaries;

- live and historical orders;
- order-status updates;
- dish management;
- menu availability;
- recommendation insights; and
- controlled recommendation tools.

The administrator pages were separated from the customer interface to prevent operational controls from appearing in the normal ordering flow. Furthermore, this separation made it easier to organize the larger number of management and research functions into dedicated sections.

3.4.3 Back-End, Security and Data Access Object Implementation

The back end was implemented as a modular Axum application. Routes define the available URLs and HTTP methods while request handlers validate input, inspect session information, call the relevant application service and return HTML or JSON responses.

The source-code structure separates into multiple responsibilities:

Component	Responsibility
Routes	Define URLs and accepted HTTP methods
Request handlers	Receive requests and return responses
Shared application state	Coordinates menu data, sessions, orders, and services
Validation functions	Check customer, order, dish, and preference inputs
Persistence modules	Read and write CSV, JSONL, and local files
Recommendation modules	Calculate scores, rankings, confidence, and explanations
Templates and static files	Produce and style the browser interface

This modular structure prevents request handlers from containing all recommendations and DAO logic. Instead, handlers coordinate the workflow and pass validated information to the appropriate module. As a result, the recommendation formulas can be modified without redesigning the customer interface or file-loading code.

Session and access implementation

Customer and administrator sessions were implemented separately. A customer first enters a name, telephone number and table identifier. Once the information is validated, the system creates a temporary session used for checkout and order tracking. Customer order requests are then checked against the active session so that one customer cannot directly retrieve another customer's order information.

Meanwhile, administrator authentication uses separately configured credentials. The username and password are loaded from environment settings rather than being embedded as default credentials in the source code. Protected administrator pages and mutation endpoints verify the administrator session before allowing operations such as updating orders or modifying dishes.

Data loading and data records

The application loads menu and historical order information from local CSV files. Dish records contain fields such as:

- dish identifier
- dish name
- ingredients
- category
- tags
- price
- availability
- image information

Historical-order records contain the order identifier, ordered dish basket and timestamp required for popularity and co-order analysis.

Before the data is used, the loader checks for issues such as:

- missing required columns;
- duplicate dish identifiers;
- invalid timestamps; and
- historical orders referencing unknown dishes.

These checks are important because incorrect identifiers or malformed order baskets could affect later recommendation calculations. Different persistence formats were used for different purposes:

- CSV stores the active menu and completes historical baskets;
- JSONL stores detailed runtime or recommendation-learning records;
- local files store dish images; and
- in-memory structures manage active sessions and live application state.

Completed order data records

Checkout creates a live operational order. However, the order is not immediately treated as confirmed collaborative evidence. Only after the administrator changes the order status to Completed is its dish basket added to the historical-order records.

The process can be summarized as follows:

1. Customer Checkout
2. Live Order Created
3. Administrator Processes Order
4. Order Marked as Completed
5. Completed Basket Added to Historical Orders
6. Future Popularity and Co-Order Evidence Updated

Lastly, pending, cancelled and incomplete orders do not influence the recommender. The system also prevents the same completed order from being appended repeatedly.

3.4.4 Recommendation Engine Implementation

The conceptual recommendation pipeline, component scores, adaptive weighting and eligibility rules were presented in Section 3.3.6. Therefore, this subsection focuses only on how the proposed design was organized and executed within the software.

The recommendation functions were implemented in dedicated Rust modules rather than inside the web request handlers. When a recommendation request is received, the handler first

validates the submitted preferences and selected dish identifiers. It then passes the validated information to the shared application state, which prepares the data required by the recommendation module. This separation ensures that HTTP handling, data persistence and recommendation calculations remain independent.

For each request, the system prepares a shared scoring context containing:

- the currently available menu items;
- the customer's liked and disliked ingredients;
- preferred tags and selected dishes;
- historical dish-pair counts;
- dish-popularity information;
- time context; and
- the adaptive weights selected for the current evidence conditions.

This context is created once and reused when evaluating all candidate dishes. Consequently, the system does not need to rebuild the historical co-order and popularity information separately for every candidate.

The implementation sequence can be summarized as follows:

1. Validated Recommendation Request
2. Create Request-Scoped Scoring Context
3. Evaluate Eligible Candidate Dishes
4. Store Component Scores and Supporting Evidence
5. Apply Deterministic Ranking and Tie-Breaking
6. Apply Selected Diversity Mode
7. Generating Recommendation Response

Each eligible dish is converted into a structured recommendation result containing its component scores, matched preferences, supporting co-order relationships, ranking position, evidence confidence and explanation data. The candidates are then ordered using deterministic

comparison rules. Therefore, the same menu, historical data, preferences and context produce the same ranking.

Moreover, diversity reranking is applied only after the initial recommendation ranking has been produced. Since ineligible dishes have already been removed, the reranking stage can change the order of suitable dishes but cannot restore an unavailable or restricted item.

Finally, explanations and confidence indicators are generated from the same structured evidence used during scoring. This prevents the displayed reason from contradicting the actual recommendation calculation. For example, a recommendation is described as co-order-supported only when relevant historical pair evidence exists.

3.5 System Testing Method

Following the completion of the main system modules, testing was conducted to verify whether the prototype behaved according to the functional and non-functional requirements defined in Section 3.2. The testing process focused on **system correctness** including whether customer actions, administrator operations, session controls, persistence functions and recommendation rules operated as intended.

Recommendation evaluation was treated separately from system testing. In this section, recommendation testing only verifies whether the implemented rules function correctly. In contrast, Section 3.6 examines how ingredient preferences, co-order evidence and different recommendation methods affect ranking outcomes. This separation prevents functional correctness from being confused with recommendation effectiveness.

3.5.1 Testing Strategy

A layered testing strategy was adopted. Firstly, individual functions and modules were checked in isolation. Next, connected modules were tested together. Finally, complete customer and administrator workflows were performed from beginning to end.

Testing level	Main purpose	Examples
Unit and component testing	Verify isolated functions and business rules	Data validation, scoring rules, hard exclusions, weight normalization
Integration testing	Verify communication between connected modules	Checkout to live order, status update to customer tracking, completed order to persistence
Functional testing	Confirm that each feature satisfies its stated requirement	Menu search, preference selection, dish management, order updates
End-to-end testing	Verify complete workflows from the user's perspective	Customer registration through order tracking; administrator login through order completion

Security testing	Verify authentication, authorisation, and role separation	Protected administrator access and customer-specific order access
Responsive interface testing	Verify usability across different screen sizes	Mobile, tablet, and desktop layouts

Table 3.7: System Testing Strategy

Synthetic customer details and controlled menu or order records were used during testing. This is to avoid the need to process genuine customer data while allowing the workflows to be repeated consistently.

Each test case was defined using:

- A test identifier
- The function or requirement being tested
- Required preconditions
- Test steps
- Expected behaviour
- Actual behaviour
- Final status

3.5.2 Unit and Component Testing

Unit and component testing focused on isolated functions that could be checked without completing the entire restaurant-ordering workflow. These tests were quite important for validation and recommendation rules where a small calculation or data error could affect several later functions.

The main component-level checks included:

1. Data and input validation

The data-loading and validation functions were tested using both valid and invalid input records. The test conditions included:

- missing required CSV columns;
- duplicate dish identifiers;
- invalid timestamps;
- historical orders containing unknown dish identifiers;
- invalid telephone-number formats;
- invalid quantities;
- unavailable dishes; and
- incomplete administrator configuration.

The expected behaviour was for invalid data to be rejected or handled safely before it entered the application state.

2. Search and preference processing

The search component was checked using dish names, ingredients, categories, tags, and dish identifiers. Preference processing was also checked to ensure that:

- liked and disliked ingredients were recognised correctly
- duplicate preference values were removed
- one ingredient could not remain both liked and disliked
- unknown preference values did not produce an application failure.

3. Recommendation-rule correctness

At the component level, recommendation tests were used to verify implementation invariants rather than compare recommendation methods. The main checks were:

- identical inputs and historical data produce identical outputs
- unavailable dishes are excluded
- already-selected dishes are excluded
- disliked ingredients always produce a hard exclusion;
- adaptive weights sum to one
- zero-history scenarios still produce a deterministic fallback
- evidence confidence remains separate from ranking scores

- diversity reranking retains only eligible candidates; and
- meal-set output remains within the specified budget and restrictions.

3.5.3 Integration Testing

After the individual components were checked, integration testing examined whether connected modules exchanged data correctly. This was necessary because the customer interface, administrator interface, recommendation engine and persistence layer share related order and menu information.

The principal integration paths were:

1. Customer checkout and live-order creation

The test began with a valid customer session and a cart containing available dishes. The checkout request was then submitted to determine whether:

1. the server validated the session
2. dish identifiers and quantities were checked
3. a live order was created
4. the order total was calculated
5. the new order became visible to the administrator

2. Administrators update and customer tracking

An existing live order was moved through its operational statuses. Subsequently, the customer tracking interface was checked to determine whether it displayed the updated status associated with the same order.

3. Completed order and historical persistence

When an order reached ***Completed***, the testing process checked whether the completed basket was transferred to historical order storage. Furthermore, repeated completion requests were tested to ensure that the same order was not appended more than once.

This integration was represented by the following sequence:

$$Customer\ checkout \rightarrow Live\ order \rightarrow Administrator\ status\ update \rightarrow Completed\ order \rightarrow Historical\ basket$$

4. Dish management and customer menu

Dish-management operations were tested to confirm whether creating, editing, deleting or changing the availability of a dish was reflected in the customer menu and recommendation candidate set.

5. Controlled-tool isolation

The research tools were integrated with the recommendation engine but were required to remain isolated from operational history. Therefore, temporary co-order simulations and counterfactual scenarios were checked to ensure that they did not modify:

- the historical-order count
- completed-order CSV records
- operational recommendation-learning records

3.5.4 Functional and End-to-End Testing

Functional testing examined the system from the perspective of its two roles: ***the customer and the administrator***. Each scenario was linked to the functional requirements identified in Section 3.2.

Customer Scenario:

The customer scenario began from an unregistered browser state and followed the complete ordering journey:

1. access the application without an active session
2. register a temporary customer session
3. browse the full menu
4. use the dish-search locator
5. select liked and disliked ingredients
6. refresh personalised recommendations
7. generate and clear a meal set
8. add dishes and quantities to the cart
9. complete checkout
10. view the submitted order through the tracking interface.

The expected outcome was a continuous customer journey without requiring access to administrator functions.

Administrator Scenario:

The administrator's scenario began from an unauthenticated state and covered the operational workflow:

1. access the administrator login page
2. authenticate using configured credentials
3. view dashboard information
4. inspect live and historical orders
5. update an order from Pending to Preparing
6. update the order to Ready
7. mark the order as Completed
8. verify the completed order persistence process
9. create, edit, disable, and remove a test dish
10. access menu export and recommendation-analysis functions.

The customer and administrator scenarios were connected through the order lifecycle. For example, the order created during customer checkout became the order processed in the administrator scenario. This method verified the complete workflow rather than testing both interfaces with unrelated records.

3.5.5 Security and Responsive Interface Testing

Security and interface testing were combined in this subsection because both relate to non-functional requirements rather than recommendation effectiveness.

Security and access-control testing

The security tests focused on the separation between customer and administrator privileges. The scenarios included:

- attempting to access an administrator page without logging in
- attempting to access an administrator API using only a customer-session cookie
- submitting invalid administrator credentials
- starting the application without configuring administrator credentials
- requesting an order that does not belong to the current customer session
- submitting unknown dish identifiers
- modifying an order through an unauthorized request
- attempting to access experimental functions without administrator authentication.

The expected behaviour was for unauthorized requests to be rejected or redirected without changing application data. Input validation was also included because malformed values may create security and data-integrity risks. Therefore, telephone numbers, quantities, dish identifiers, order identifiers, preferences and status values were checked before being passed to the relevant service.

Responsive interface testing

The customer and administrator interfaces were inspected at mobile, tablet and desktop viewport sizes. The main interface checks were:

- navigation controls remained accessible
- menu cards did not overlap
- dish names and prices remained readable
- cart quantities and prices remained aligned

- preference controls remained selectable
- modal content stayed within the viewport
- recommendation cards remained navigable
- administrator order controls remained usable
- no essential information required horizontal page scrolling

Mobile testing received greater attention because customers are expected to access the ordering interface through their own phones. Nevertheless, the administrator interface was also checked on wider screens because its order, dish and analysis views contain more operational information.

3.6 Recommendation Evaluation Method

While Section 3.5 verifies whether the system functions correctly, this section explains how the recommendation approaches were evaluated. The evaluation focused on three observable effects which are ***the influence of explicit ingredient preferences, historical co-order evidence and the performance differences among ingredient-only, co-order-only, and hybrid recommendation methods.***

3.6.1 Evaluation Design and Experimental Controls

Each experiment was aligned with one research question.

Research question	Controlled experiment	Main evaluation focus
RQ1: How do liked and disliked ingredient preferences affect dish ranking and restriction compliance?	Ingredient Preference Impact	Rank movement, preference matches and exclusions
RQ2: How does increasing co-order evidence affect association measures and recommendation ranking?	Co-Order History Impact	Pair evidence, association measures and rank movement
RQ3: How do ingredient-only, co-order-only, and hybrid methods compare in recovering a hidden dish?	Recommendation Method Comparison	Hit@K and hidden-dish rank

Table 3.8: Alignment of Research Questions and Experiments

To ensure that the comparisons remained fair and repeatable, the following controls were applied:

- the same menu dataset was used throughout each comparison
- the same historical-order baseline was retained
- the Top-K value remained fixed within each experiment
- only the variable being investigated was changed
- disliked-ingredient exclusions remained active

- simulated baskets were created temporarily in memory
- simulated data were not written to the operational order history
- identical inputs produced identical outputs

Therefore, the experiments measured recommendation behaviour under controlled conditions rather than attempting to reproduce the continuously changing conditions of a live restaurant.

3.6.2 Ingredient Preference Impact Experiment

The Ingredient Preference Impact experiment examined how explicit likes and dislikes changed the recommendation ranking.

First, the system generated a neutral baseline ranking without selected ingredient preferences. In addition, liked and disliked ingredients were applied while the menu, historical orders, contextual dishes and Top-K value remained unchanged. The two rankings were then compared.

The procedure was:

1. select the experiment scenario and Top-K value
2. generate the neutral recommendation ranking
3. apply the selected liked and disliked ingredients
4. generate the preference-shaped ranking
5. compare the position of each candidate dish
6. identify preference matches and hard exclusions

Rank movement was calculated as:

$$\Delta \boldsymbol{Rank_i} = \boldsymbol{Rank_{baseline,i}} - \boldsymbol{Rank_{preference,i}}$$

where:

- $\Delta Rank_i$ represents the change in position of dish i;
- $Rank_{baseline,i}$ is its position before preferences were applied; and

- $Rank_{preference,i}$ is its position after preferences were applied.

A positive value indicates that the dish moved upward whereas a negative value indicates that it moved downward. In addition, a dish containing a disliked ingredient was recorded as excluded instead of being assigned a lower ranking.

Through of this experiment, we able to observe whether compatible dishes moved direction, which ingredients contributed to the recommendation and which of the disliked-ingredient violations occurred.

3.6.3 Co-Order History Impact Experiment

The Co-Order History Impact experiment was conducted to examine whether stronger historical co-order evidence could influence the relationship between two dishes and subsequently change the position of a candidate dish in the recommendation ranking.

For each test, one dish was selected as the **anchor dish** while another was selected as the **candidate dish**. The system first calculated the existing relationship between the two dishes using the original historical-order dataset. This produced the baseline pair count, association measures, co-order score, and ranking position of the candidate.

After the baseline had been recorded, a selected number of temporary order baskets containing both the anchor and candidate dishes was added to an in-memory copy of the historical data. The relationship measures and recommendation ranking were then recalculated using the simulated dataset. Since the menu, preferences, anchor dish, candidate dish and other experimental conditions remained unchanged, the difference between the baseline and simulated results could be attributed mainly to the additional co-order evidence.

The comparison focused on changes in the pair count, support, association confidence, lift, co-order component score and candidate rank. An increase in these values would indicate that repeated co-occurrence strengthened the behavioural relationship between the two dishes. Nevertheless, a stronger relationship did not automatically guarantee that the candidate would become the highest-ranked dish because the ranking could still be affected by the relative strength of other eligible candidates.

Importantly, the temporary baskets were created only for the duration of the experiment. They were not appended to the operational historical-order file or treated as completed customer orders. This means the same experiment could be repeated without permanently changing later recommendation results.

3.6.4 Recommendation Method Comparison

The Recommendation Method Comparison experiment was designed to compare the behaviour of the two individual recommendation approaches with a controlled hybrid configuration. Three methods were evaluated: ***ingredient-only, co-order-only and fixed hybrid recommendation.***

Method	Ingredient weight	Co-order weight
Ingredient-only	1.0	0.0
Co-order-only	0.0	1.0
Fixed hybrid	0.4	0.6

Table 3.9: Controlled Recommendation Methods

Fixed weights were used because the purpose of this experiment was to compare the recommendation methods under equivalent conditions. The adaptive production weights were therefore not applied as changing the weights according to each scenario would make it more difficult to identify whether the difference was caused by the recommendation method or by the adaptive weighting process.

A leave-one-item-out procedure was used. First, a historical order containing at least two dishes was selected. One dish was then hidden and treated as the target item while the remaining dishes were used as the known ordering context. Each of the three methods generated its own ranked recommendation list from the same context after the system recorded whether the hidden dish was recovered and where it appeared in the ranking.

For example, assume that a historical order contained the following dishes:

$$\{D01, D05, D08\}$$

If $D08$ was selected as the hidden dish, $D01$ and $D05$ would be provided as the known context. The ingredient-only method would rank candidates using content evidence whereas the

co-order-only method would rely on historical dish relationships. The fixed hybrid method would then combine both signals using the predetermined weights.

The comparison did not assume that one method would perform best in every situation. Ingredient-only recommendations may be more useful when relevant preference information is available but behavioural data are limited. Conversely, co-order recommendation may perform better when the context dishes have strong historical relationships. The hybrid configuration was intended to examine whether combining the two signals could recover the hidden dish more consistently under the selected cases.

3.6.5 Evaluation Metrics and Interpretation

Several metrics were used because recommendation behaviour cannot be described adequately through a single measurement. Rank-based metrics were used to evaluate position changes and hidden-dish recovery while association measures were used to interpret the strength of co-order evidence.

1. Rank movement

Rank movement measures the change in the position of a dish before and after the experimental condition is applied:

$$\Delta Rank_i = Rank_{before,i} - Rank_{after,i}$$

A positive value means that the dish moved closer to the top of the ranking whereas a negative value indicates that it moved downward. If a dish was removed because of a disliked ingredient, it was recorded as excluded rather than assigned a numerical rank.

2. Hit@K

Hit@K indicates whether the hidden dish appears within the first Krecommendations:

$$Hit@K = \begin{cases} 1, & if\ the\ hidden\ dish\ appears\ within\ the\ Top-K \\ 0, & otherwise \end{cases}$$

For example, Hit@3 is equal to one when the hidden dish appears in the first, second or third position. Although this metric shows whether recovery was successful within the selected recommendation range, it does not show the exact position of the dish.

3. Hidden-dish rank

Hidden-dish rank records the precise position of the target dish in the recommendation list. A lower rank represents a better recovery position. Therefore, Hit@K and hidden-dish rank were interpreted together. Two methods may both achieve Hit@3, but a method that ranks the hidden dish first performs differently from one that ranks it third.

4. Preference match rate

Preference match rate measures the proportion of recommended dishes that match at least one selected preference:

$$PreferenceMatchRate = \frac{Number\ of\ recommendations\ matches\ the\ selected\ preferences}{Total\ number\ of\ recommendations}$$

This metric was used only when explicit preference information was provided. A higher match rate indicates stronger alignment with the selected ingredients or tags, although it does not by itself measure customer satisfaction.

5. Restriction violations

Restriction violations measure whether any recommended dish contains an ingredient that was marked as disliked:

$$RestrictionViolations = \sum_{i=1}^{K} Violation_i$$

The expected value was zero because disliked ingredients were treated as hard exclusions. Therefore, this metric served primarily as a compliance measure rather than as a comparative accuracy score.

6. **Support**

Support measures how frequently the anchor and candidate dishes occur together across the complete set of historical baskets. For anchor dish A, candidate dish B and Nhistorical baskets, support is calculated as:

$$Support(A, B) = \frac{count(A, B)}{N}$$

A higher support value indicates that the selected pair appears more frequently in the overall order history.

7. Association confidence

Association confidence measures the proportion of baskets containing the anchor dish that also contains the candidate dish:

$$Confidence(A \rightarrow B) = \frac{count(A, B)}{count(A)}$$

Unlike pair count, confidence is directional. Therefore, the confidence of $A \rightarrow B$ may differ from the confidence of $B \rightarrow A$, especially when one dish appears much more frequently than the other.

8. Lift

Lift compares the observed co-order relationship with the frequency that would be expected from the general popularity of the candidate dish:

$$Lift(A \rightarrow B) = \frac{Confidence\ (A \rightarrow B)}{count(B)/N}$$

A lift greater than one indicates that the dishes occur together more frequently than expected based on the candidate's overall frequency. A value near one suggests that the relationship is broadly consistent with general popularity while a value below one indicates that the dishes occur together less frequently than expected.

However, support, confidence and lift represent association rather than causation. They can show that two dishes frequently appear together, but they cannot prove that selecting one dish causes a customer to order the other.

3.7 Ethical and Responsible-System Considerations

Although the system does not create permanent customer accounts, it collects a customer name, telephone number and table identifier to support checkout and order tracking. Therefore, the collected information should not be described as fully anonymous. The principle of data minimization recommends collecting only information that is relevant and necessary for a specified purpose, particularly in personalized information systems (Biega et al., 2020).

Nevertheless, local storage alone does not ensure customer privacy. A real restaurant deployment would require defined retention periods, controlled staff access, secure storage and clear communication regarding the use of customer information. Privacy, accountability, transparency, reliability and the management of harmful bias are also recognized as important characteristics of trustworthy computational systems (National Institute of Standards and Technology [NIST], 2023).

User autonomy was preserved by treating recommendations as optional decision support. Customers remain able to browse the complete menu, change or clear their preferences, ignore suggested dishes and manually choose other available items. Hence, the recommendation engine assists menu exploration without replacing the customer's final decision.

Moreover, recommendation transparency was considered because ranked suggestions may influence customers' attention and ordering choices. Explainable recommendation aims to provide understandable reasons alongside recommendation outputs instead of presenting rankings without justification (Zhang & Chen, 2020). Therefore, the prototype displays information such as matched preferences, co-order evidence, and plain-language reasons where applicable.

Overall, the application is treated as an explainable research prototype and restaurant decision-support system rather than a medical, nutritional or statistically predictive system. Its limitations are therefore disclosed, customer choice is retained and evaluation claims are restricted to evidence produced under the controlled experimental conditions.

Chapter 4: Result and Discussion

4.1 Chapter Overview

This chapter presents the results obtained from the implementation, testing and controlled evaluation of the **Personalized Restaurant Ordering System**. In contrast to Chapter 3, which explained how the system was designed, developed and evaluated, the present chapter focuses on the observable evidence produced by those activities. Therefore, architectural descriptions, recommendation formulas, implementation procedures and experimental steps are not repeated unless they are necessary for interpreting a specific result.

The results are presented in three stages. First, the completed system outcomes are demonstrated through selected customers, administrator and recommendation-tool interfaces. These outcomes are then linked to the functional and non-functional requirements established in Section 3.2. This provides evidence that the developed artefact covers both the operational restaurant workflow and the recommendation-related research functions described in the project methodology

Secondly, the chapter reports the actual system-testing outcomes. These results cover component correctness, communication between connected modules, complete customer and administrator workflows, access control, data persistence and responsive interface behaviour. The purpose of this stage is to determine whether the system operated according to its specified requirements rather than to assess which recommendation method performed best.

Last of not least, the controlled recommendation results are then presented according to the three research questions. The Ingredient Preference Impact experiment addresses RQ1 by examining changes caused by liked and disliked ingredients. The Co-Order History Impact experiment addresses RQ2 by measuring how additional behavioural evidence changes dish associations and ranking positions. Finally, the Recommendation Method Comparison addresses RQ3 by comparing ingredient-only, co-order-only, and fixed-hybrid methods using hidden-dish recovery.

4.2 Implemented System Outcomes

This section presents the observable outcomes of the completed **Personalized Restaurant Ordering System**. Rather than repeating the design and implementation details discussed in Chapter 3, the section uses selected interface evidence to demonstrate the customer ordering workflow, administrator management functions and recommendation-analysis feature available in the final prototype.

The completed application integrates the main restaurant-ordering activities within a browser-based environment. Customers can register a temporary restaurant session, browse the menu, provide preferences, receive recommendations, manage a cart, submit an order and monitor its status. At the same time, administrators can access a separately protected interface for order processing, dish management, operational insights and recommendation testing. The current implementation also includes the three controlled tools required for examining ingredient impact, co-order impact and recommendation-method differences.

4.2.1 Customer-Side System Outcome

The customer interface provides a continuous ordering journey beginning with temporary session registration. The customer enters a name, Malaysian telephone number and table identifier before accessing the main ordering interface. This allows checkout and later order tracking to be linked to the current restaurant session without requiring a permanent customer account.

After entering the system, the customer can browse the complete menu using category navigation. In addition, the search function allows a dish to be located through its name, category, ingredients, tags or identifier. The search result directs the customer to the matching menu item while retaining the full menu, allowing other dishes to remain available for comparison.

Preference controls are presented within the same customer interface. Customers may select liked ingredients, disliked ingredients, preferred tags, and context dishes based on the vocabulary available on the menu. As these selections are changed, the recommendation section displays results that correspond to the current preference and ordering context.

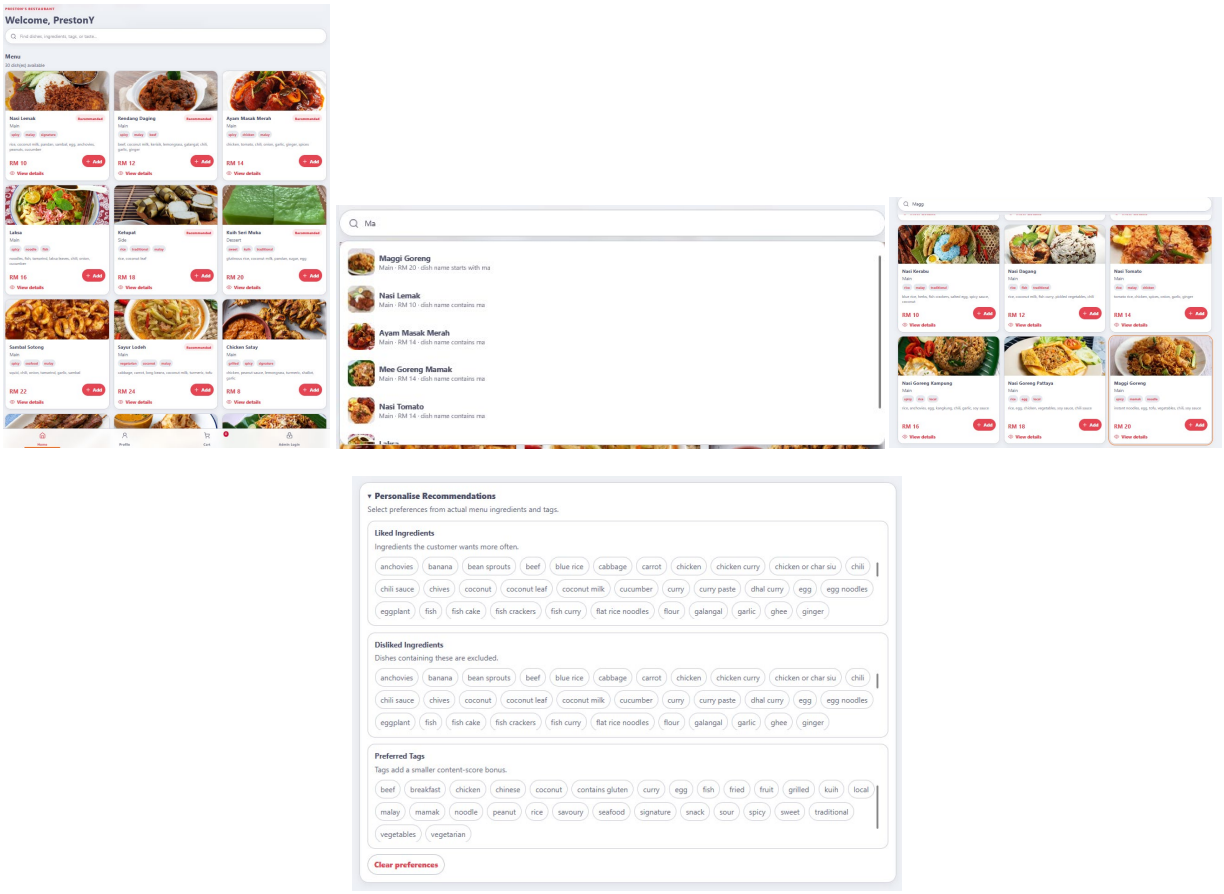


Figure 4.1: Customer Menu, Search and Preference-Selection Interfaces

As shown in Figure 4.1, the customer interface combines menu browsing, search and preference input within the same ordering page. The full menu remains accessible after a search is conducted while preference controls allow the recommendation request to be adjusted without requiring a separate customer-profile page. Therefore, the implemented interface supports both customers who wish to browse manually and those who prefer guided dish discovery.

The recommendation outcome is displayed through the **Recommended for You** section. Each result presents the recommended dish together with a plain-language reason and an indication of the evidence supporting the suggestion. Moreover, customers can inspect additional information such as matched preferences and component evidence rather than receiving only an unexplained ranking.

The customer can also select among the Familiar, Balanced and Discover modes. These modes allow the recommendation presentation to range from closer preference alignment to broader menu exploration. In addition, the meal-set function generates a combination of dishes

according to the selected budget, party size, category needs and active restrictions. The repository confirms that the implemented customer experience includes explanations, diversity modes, and budget-aware meal-set suggestions.

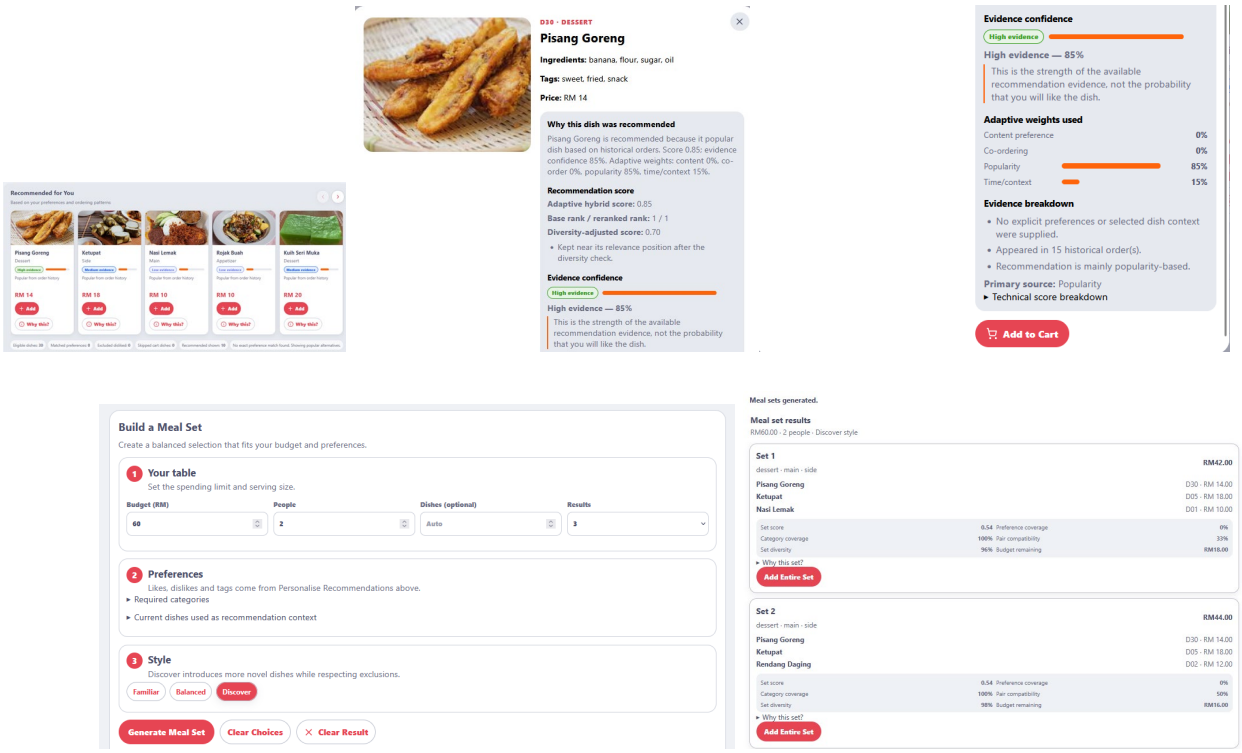


Figure 4.2: Personalized Recommendation and Meal-Set Outcomes

Figure 4.2 demonstrates that the recommendation outcome is integrated directly into the customer ordering interface. Besides displaying ranked dishes, the interface provides supporting reasons and evidence indicators. The generated meal set further extends the recommendation function from individual dishes to a suggested combination that follows the customer's selected budget and restrictions.

Once a suitable dish has been selected, the customer can add it to the cart, adjust the quantity, remove items and review the subtotal. The checkout interface then submits the selected dishes as a restaurant order. After submission, the order appears in the customer's tracking area where its status can be viewed as the administrator processes it.

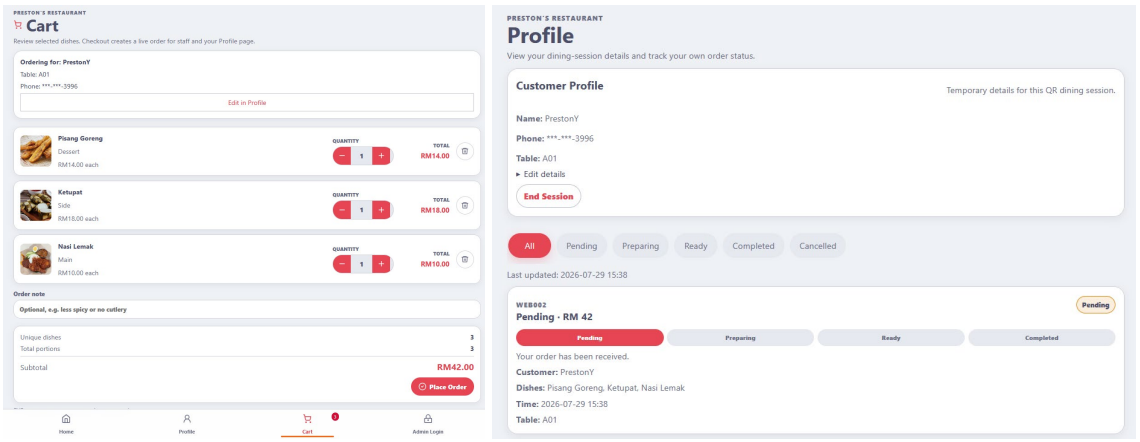


Figure 4.3: Customer Cart, Checkout and Order-Tracking Outcomes

As illustrated in Figure 4.3, the customer-side outcome covers the complete workflow from dish selection to order monitoring. The cart provides quantity and subtotal information before checkout while the tracking interface displays the operational status of the submitted order. Hence, the recommendation functions are embedded within a functioning ordering workflow rather than being presented as an isolated demonstration.

4.2.2 Administrator-Side System Outcome

The administrator interface is separated from the customer ordering pages and requires its own authentication. Once access is granted, the administrator dashboard presents information relevant to restaurant operations, including active orders, historical orders, dish popularity, and commonly observed co-order relationships.

The live-order view allows several customer orders to be inspected without replacing the previous order on the screen. Furthermore, each order can be moved through the available operational states: ***Pending, Preparing, Ready, Completed,*** or ***Cancelled***. When an order is updated, the revised status becomes available to the corresponding customer tracking interface.

Dish-management functions are also available from the administrator area. Administrators can create new dishes, edit existing dish information, change availability, delete dishes, preview images and export menu information. In comparison with the customer interface, these controls focus on maintaining the operational menu rather than supporting dish selection.

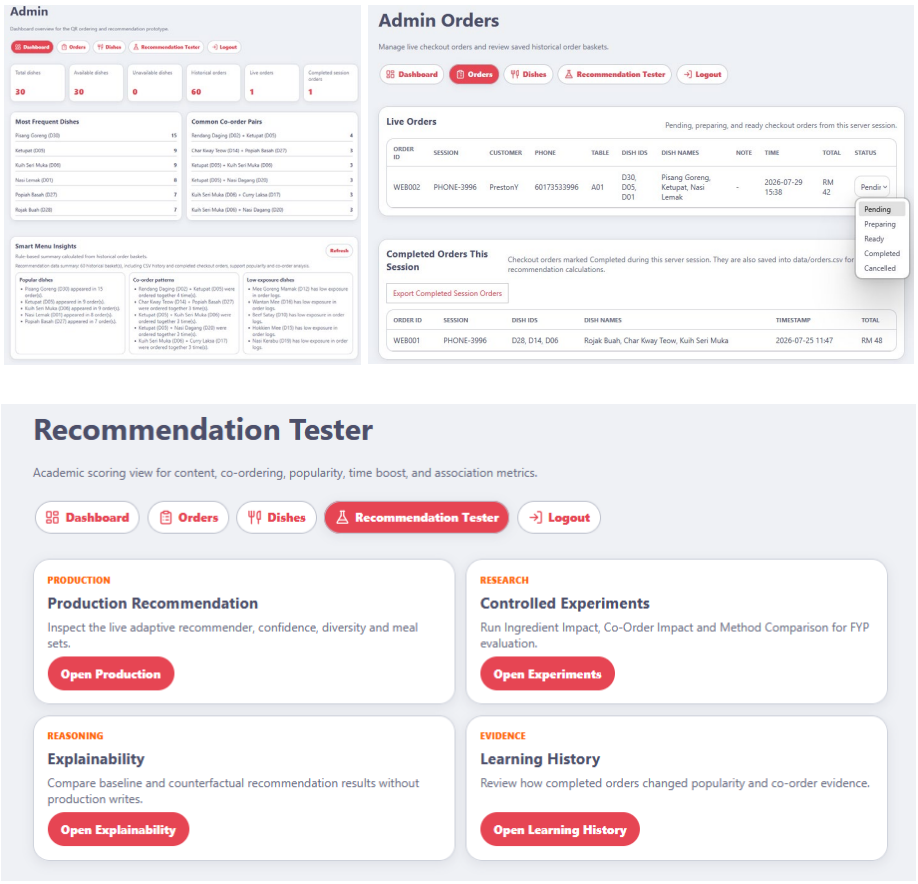


Figure 4.4: Administrator Management and Recommendation-Testing Interfaces

Figure 4.4 shows the separation between operational and research-oriented administrator functions. The order board supports the processing of live customer orders whereas the dish-management page supports menu maintenance. Last but not least, the Recommendation Tester provides a dedicated environment for examining recommendation behaviour without placing detailed experimental controls in the customer ordering interface.

4.2.3 Recommendation and Research-Tool Outcome

The completed Recommendation Tester contains both production-oriented analysis and controlled research tools. Production recommendation analysis allows the administrator to inspect adaptive scoring, evidence confidence, diversity behaviour, meal sets, temporary counterfactual changes and the learning timeline.

In addition, three controlled experiment modules were implemented:

1. Ingredient Preference Impact
2. Co-Order History Impact
3. Recommendation Method Comparison

The Ingredient Preference Impact tool displays how selected likes and dislikes affect ranking positions and exclusions. The Co-Order History Impact tool demonstrates how temporary pair evidence changes association measures and candidate ranking. Meanwhile, the Method Comparison tool evaluates ingredient-only, co-order-only, and fixed-hybrid methods through hidden-dish recovery. These tools are presented as separate administrator functions rather than as customer-facing features.

4.2.4 Summary of Implemented Outcomes

Table 4.1 summarizes the principal functions demonstrated by the completed prototype.

System area	Main implemented outcomes	Implementation status
Customer ordering	Session registration, menu browsing, search, preferences, cart, checkout and order tracking	Implemented
Personalised recommendation	Ingredient and context inputs, ranked suggestions, explanations, evidence indicators and diversity modes	Implemented
Meal-set generation	Budget, party-size, category and restriction-aware dish combinations	Implemented
Administrator management	Authentication, dashboard, live and historical orders, status updates and dish management	Implemented
Operational insights	Dish popularity, named co-order pairs and recommendation evidence	Implemented
Research tools	Ingredient Impact, Co-Order Impact and Method Comparison	Implemented

Table 4.1: Summary of Implemented System Outcomes

4.3 System Testing Results

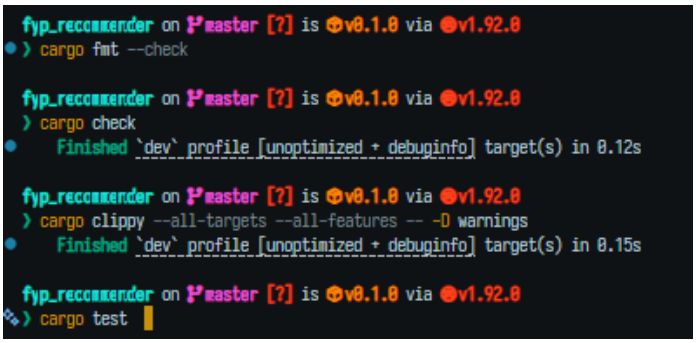
This section presents the results of the system testing procedures described in Section 3.5. While the previous section demonstrated the functions available in the completed prototype, the present section reports whether those functions behaved correctly under the specified test conditions. Recommendation quality and method comparison are excluded from this section because they are examined separately in Section 4.4.

Testing was carried out on a Windows local development environment using ***Rust 1.92.0*** and ***Chromium*** browser automation through ***Playwright***. The test dataset contained **30** dishes and **60** initial historical-order baskets. A disposable copy of the data files was used for browser testing so that checkout and completion tests could exercise genuine CSV persistence without modifying the original project dataset.

4.3.1 Overall Testing Results

The automated quality gate included formatting validation, compilation checking, strict linting and the Rust test suite. All four commands completed successfully:

```
cargo fmt --check
cargo check
cargo clippy --all-targets --all-features -- -D warnings
cargo test
```



The Rust test suite reported **115 tests passed, 0 failed, and 0 ignored**. In addition, 55 report-level checks were selected to represent the main unit, integration, end-to-end, security and responsive interface requirements.

Test category	Tests conducted	Passed	Failed	Pass rate

Unit and component testing	12	12	0	100%
Integration testing	8	8	0	100%
End-to-end workflow testing	3	3	0	100%
Security and access-control testing	8	8	0	100%
Responsive-interface inspection	24	24	0	100%
Total	55	55	0	100%

Table 4.2: Overall System Testing Results

All 55 selected checks passed, producing an overall pass rate of 100%. The 115 automated Rust tests and the 55 report-level checks should not be treated as the same measurement. The ***Rust suite*** examines lower-level implementation behaviour whereas the report-level cases summaries the system functions most relevant to the project requirements.

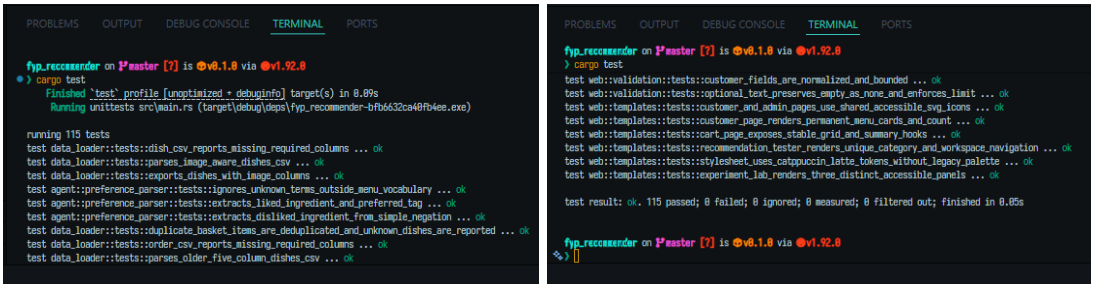


Figure 4.5: Automated Rust Test Execution Result

4.3.2 Unit and Component Test Results

The unit and component tests examined isolated validation, search, preference, recommendation and meal-set functions. Table 4.3 presents the main test groups. Besides that, the full list of individual cases will be placed in the appendix.

Test area	Main condition tested	Result

Dataset validation	Missing columns, duplicate dish IDs, invalid timestamps, and unknown dish references	Invalid records were rejected before entering the application state
Search processing	Search term laksa	Laksa, Curry Laksa, and Asam Laksa were returned while the full menu remained unchanged
Preference conflict	The same ingredient selected as liked and disliked	The earlier state was cleared, preventing conflicting preference values
Eligibility restrictions	Unavailable dishes and dishes containing disliked ingredients	Restricted dishes were excluded from eligible recommendations
Recommendation consistency	Weight normalisation, deterministic ranking, and diversity reranking	Weights totalled 1.0, identical inputs returned the same order, and no excluded dishes were restored

Table 4.3: Summary of Unit and Component Test Results

All 12 selected units and component cases passed. The results show that malformed data were stopped at the loading boundary and did not enter later recommendation calculations. Moreover, the hard-exclusion rules remained active during ranking and diversity reranking. The adaptive weights also remained normalized while identical inputs produced repeatable outputs. Do take note that these findings establish the correctness of the implemented rules. They do not indicate whether a particular recommendation method performs better which remains the purpose of the experiments in Section 4.4.

4.3.3 Integration and End-to-End Workflow Results

Integration testing examined whether customer ordering, administrator processing, status tracking and historical-order data operated as one connected workflow. A synthetic customer named **FYP Test Customer** registered at table **T08** and submitted Nasi Lemak and Chicken Satay,

producing an order total of **RM18.00**. The checkout created live order WEB001 with an initial status of ***Pending***. The same order then appeared on the administrator order board with its customer, table, dishes, time, total and status information. After the administrator changed the order to Preparing, the customer Profile page displayed the revised status. The order was later moved through Ready and Completed. Completion created historical record O061 and increased the historical dataset from 60 to 61 valid baskets. Repeating the completion request did not add another record and the newly appended basket remained available after the server was restarted.

Test area	Expected outcome	Actual outcome	Status
Checkout to live order	Valid checkout creates a live order	WEB001 was created with Pending status and the correct RM18.00 total	Pass
Live order to administrator	Submitted order appears on the administrator board	WEB001 appeared with the correct customer, table, dish and status details	Pass
Administrators update to customer	Status change appears in customer tracking	Customer Profile changed from Pending to Preparing	Pass
Completion to historical data	Completed basket is saved as historical evidence	O061 was appended and the valid history increased from 60 to 61 baskets	Pass
Repeated completion	The basket is not written twice	The historical dataset remained at 61 baskets	Pass
Restart persistence	Completed basket remains after restart	O061 was successfully reloaded from CSV	Pass
Dish availability integration	Disabled dish is removed from ordering and recommendation	Unavailable dish was excluded from both menu access and recommendation candidates	Pass
Experiment isolation	Temporary experiments do not alter operational history	Historical-order and learning-timeline counts remained unchanged	Pass

Table 4.4: Integration and End-to-End Workflow Results

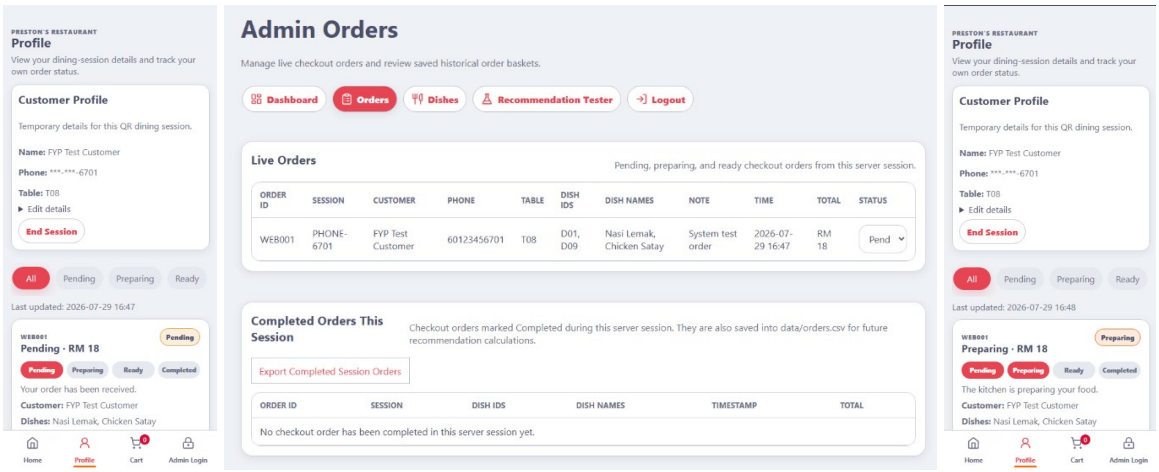


Figure 4.6: Customer Ordering and Administrator Status-Update Test

Figure 4.6 demonstrates that the customer and administrator interfaces operated on the same live order. The order submitted by the customer appeared on the administrator board while the administrator's status update was reflected on the customer Profile page. In addition, completion testing confirmed that the order basket entered historical evidence only once and remained available after restarting the application.

The end-to-end customer and administrator scenarios also passed in full. The customer completed the journey from temporary registration to order tracking, while the administrator authenticated, processed the same order and completed its operational lifecycle. These results were obtained in a local single-server environment and should not be interpreted as evidence of performance under simultaneous restaurant-scale traffic.

4.3.4 Security and Access-Control Results

Security testing focused on session separation and the rejection of unauthorized or invalid operations. Eight security scenarios were executed and passed all of it.

Test condition	Actual outcome	Status
Administrator page accessed without login	HTTP 302 redirect to /admin/login	Pass
Invalid administrator credentials	HTTP 401 response with an invalid-credentials message	Pass

Customer cookie used for administrator API	Request rejected with Admin login required	Pass
Logged-out administrator reused an old session	Previous cookie could no longer access protected pages	Pass
Customer requested another session's order	Request rejected because the order was outside the current session	Pass
Unknown dish identifier submitted	Checkout rejected the request without creating an order	Pass
Invalid order status submitted	Administrator API rejected the unknown status	Pass
Experiment endpoint accessed without administrator login	Protected request was rejected	Pass

Table 4.5: Security and Access-Control Results

The results confirmed that customer and administrator sessions remained separate and that invalid protected operations did not modify application data. For example, logging out invalidated the earlier administrator session while a customer cookie could not be reused to reach administrator APIs. These outcomes demonstrate suitable access separation for the prototype. However, the tests do not constitute a production security certification. Measures such as HTTPS termination, password hashing, CSRF protection, rate limiting and multiple staff roles remain outside the present implementation.

4.3.5 Responsive-Interface Inspection Results

Responsive inspection was conducted at three representative viewport sizes:

- mobile: **390 × 844 pixels**
- tablet: **768 × 1024 pixels**
- desktop: **1440 × 900 pixels**

Eight interface areas were inspected at each viewport, resulting in 24 checks. All 24 passed. Browser measurements also confirmed that the tested pages did not introduce full-page horizontal

overflow. Where wide content was necessary, such as recommendation rails or administrator tables, scrolling remained limited to the relevant component rather than widening the complete page.

Interface area	Mobile	Tablet	Desktop	Main observation
Customer navigation	Pass	Pass	Pass	Bottom navigation remained visible and accessible
Menu cards	Pass	Pass	Pass	Cards adapted to the available screen width
Preference controls	Pass	Pass	Pass	Options wrapped without creating page overflow
Recommendation cards	Pass	Pass	Pass	Cards remained readable within a horizontally scrollable rail
Cart and checkout	Pass	Pass	Pass	Quantities, total, and checkout action remained visible
Order tracking	Pass	Pass	Pass	Profile, status, dish and progress information remained readable
Administrator order board	Pass	Pass	Pass	Dense order information remained within its own responsive wrapper
Dish management	Pass	Pass	Pass	Search, records and actions remained accessible

Table 4.6: Responsive-Interface Inspection Results

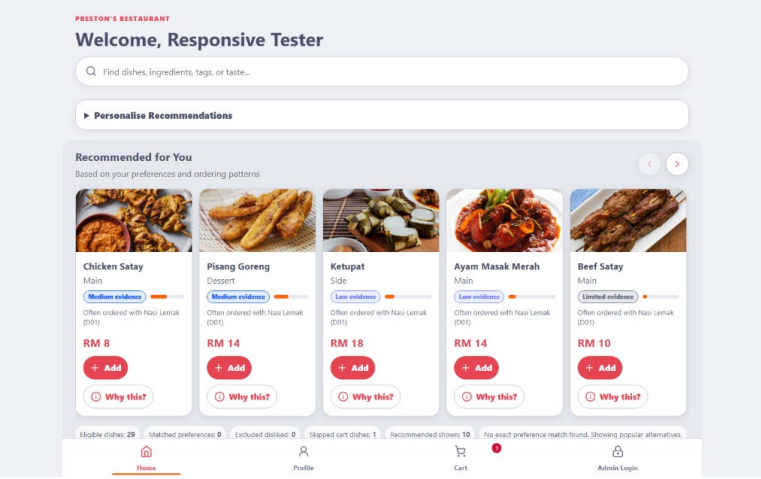


Figure 4.7.1: Desktop Viewport

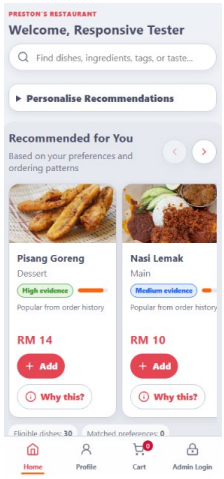


Figure 4.7.2: Mobile Viewport

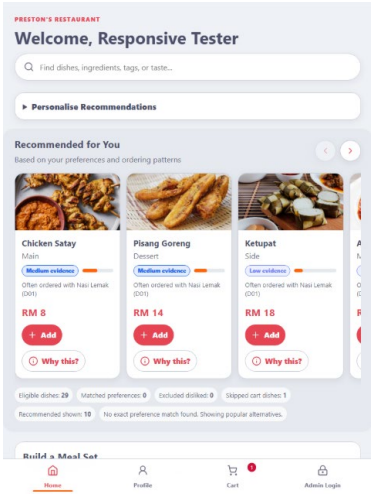


Figure 4.7.3: Tablet Viewport

As shown in Figure 4.7, the customer interface adjusted its content arrangement according to the available width. The mobile layout displayed a smaller number of recommendation cards while maintaining local horizontal navigation. The tablet view increased the visible content area whereas the desktop layout presented additional cards without removing the same functions. Across all three sizes, the search bar, recommendation controls, dish information, cart indicator and bottom navigation remained accessible.

4.3.6 Summary of Testing Findings

Overall, the automated Rust suite passed all **115 tests** while all **55 selected report-level checks** were also successful. The results confirmed that the main customer and administrator workflows operated correctly, order statuses were synchronized between the two interfaces and completed baskets had persisted without duplication. Furthermore, invalid data were rejected, protected functions required the appropriate session and temporary experiments remained isolated from operational history. The responsive inspection also showed that the principal customer and administrator functions remained accessible across the tested mobile, tablet and desktop sizes. Even so, the findings remain limited to a local single-server prototype using synthetic customer information. The testing did not include concurrent load testing, public-network deployment, advanced penetration testing or formal usability evaluation with external restaurant users.

4.4 Recommendation Evaluation Results

This section presents the results of the three controlled recommendations. The experiments were conducted on 29 July 2026 using the same menu of 30 dishes and a baseline dataset of 60 historical order baskets. Temporary co-order baskets were processed in memory and were not added to the operational order history. In addition, the experiment interface confirmed that the production recommendation weights and stored data remained unchanged throughout the tests.

4.4.1 Ingredient Preference Impact Results

The Ingredient Preference Impact experiment addressed RQ1 by comparing a neutral ranking with a ranking generated after liked and disliked ingredients were applied. Three scenarios were used to examine positive preference matching, hard exclusion and the simultaneous use of liked and disliked ingredients.

Scenario	Preference input	Selected dish	Baseline rank	Preference rank	Rank change	Outcome
IP-01	Liked: bean sprouts	Char Kway Teow (D14)	10	1	+9	Moved upward
IP-02	Disliked: banana	Pisang Goreng (D30)	1	Excluded	N/A	Hard excluded
IP-03	Liked: rice; disliked: banana	Ketupat (D05)	2	1	+1	Moved upward, D30 excluded

Table 4.7: Ingredient Preference Impact Results

In Scenario IP-01, Char Kway Teow moved from rank **10** to rank **1** after bean sprouts was selected as a liked ingredient. This represented an upward movement of nine positions while the dish received an ingredient score of **0.14**.

Scenario IP-02 produced a different outcome. Pisang Goreng was originally ranked first in the neutral result. Once banana was marked as disliked, the dish was completely removed from the eligible recommendation list rather than merely being assigned a lower rank.

The combined scenario, IP-03, applied rice as a liked ingredient and banana as a disliked ingredient. Ketupat moved from rank 2 to rank 1 with an ingredient score of 0.50. At the same time, Pisang Goreng remained excluded because it contained the disliked ingredient. Rice-based dishes such as Nasi Dagang, Nasi Goreng Pattaya, and Nasi Kandar also entered the Top-5 result.

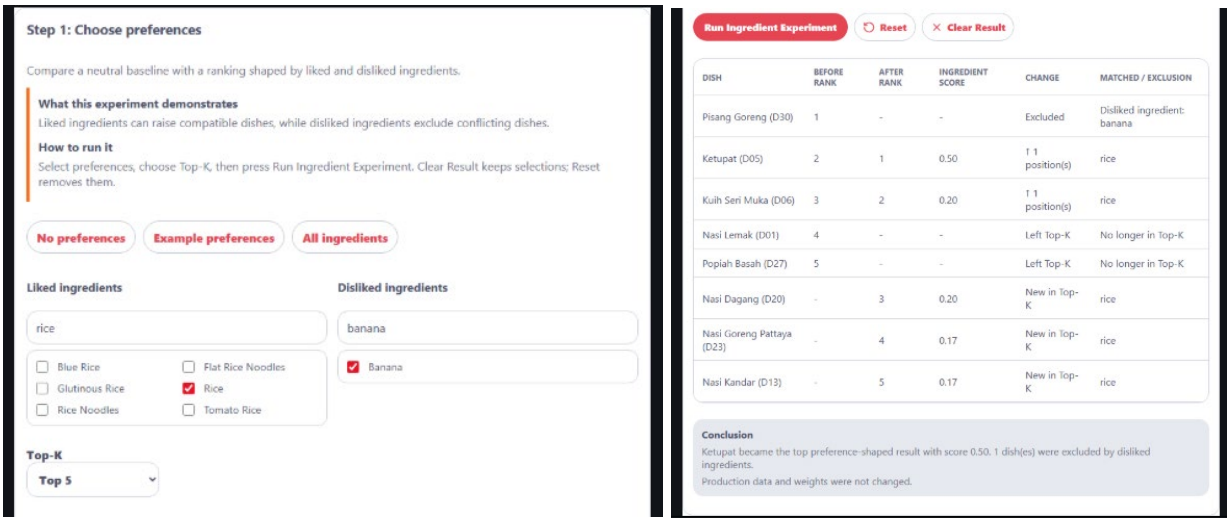


Figure 4.8: Ingredient Preference Impact Experiment Result

Figure 4.8 presents the combined liked-rice and disliked-banana scenario. Ketupat became the highest-ranked eligible dish while Pisang Goreng was excluded. The result also shows the dishes that entered or left the Top-5 after the preferences were applied.

The results answer RQ1 by demonstrating that liked ingredients can improve the relative position of compatible dishes whereas disliked ingredients operate as hard eligibility restrictions instead of weak negative preferences.

4.4.2 Co-Order History Impact Results

The Co-Order History Impact experiment addressed RQ2 by examining how additional temporary co-order baskets affected the relationship between an anchor dish and a candidate dish. Two candidate pairs were tested using zero, three and five added co-orders.

Anchor and candidate pair	Added co-orders	Pair count	Support	Association confidence	Lift	Candidate rank
Nasi Lemak (D01) → Sambal Sotong (D07)	**0**	**1**	**0.02**	**0.12**	**1.87**	**7**
Nasi Lemak (D01) → Sambal Sotong (D07)	**3**	**4**	**0.06**	**0.36**	**3.27**	**1**
Nasi Lemak (D01) → Sambal Sotong (D07)	**5**	**6**	**0.09**	**0.46**	**3.33**	**1**
Nasi Lemak (D01) → Chicken Satay (D09)	**0**	**3**	**0.05**	**0.38**	**4.50**	**1**
Nasi Lemak (D01) → Chicken Satay (D09)	**3**	**6**	**0.10**	**0.55**	**4.30**	**1**
Nasi Lemak (D01) → Chicken Satay (D09)	**5**	**8**	**0.12**	**0.62**	**4.00**	**1**

Table 4.8: Co-Order History Impact Results

For the Nasi Lemak and Sambal Sotong pair, the baseline pair count was one. After three temporary co-orders were added, the count increased to four, support increased from 0.02 to 0.06 and association confidence increased from 0.12 to 0.36. Sambal Sotong also moved from rank 7

to rank 1. After five temporary co-orders, the pair count reached six. Support increased further to 0.09, association confidence reached 0.46 and lift increased from 1.87 to 3.33. However, the candidate remained at rank 1 because it had already reached the highest possible position after the first three additions.

The Nasi Lemak and Chicken Satay pair showed a different ranking pattern. Chicken Satay was already ranked first at baseline. Although the pair count increased from three to eight and association confidence increased from 0.38 to 0.62, its ranking could not improve further. The lift value changed from 4.50 to 4.00 as the temporary additions affected both the pair frequency and the overall frequency of the candidate.

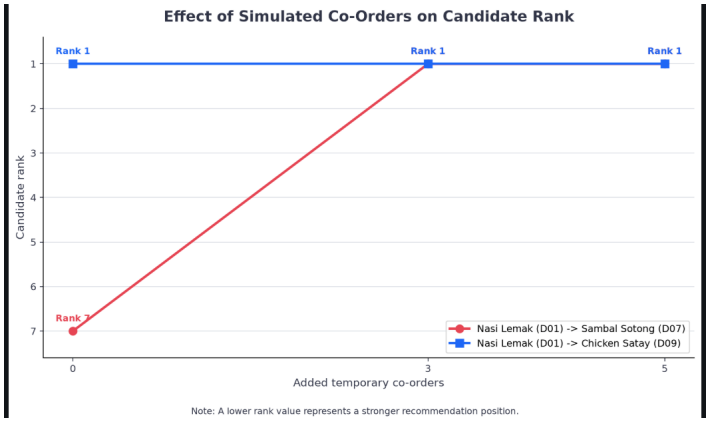


Figure 4.9: Effect of Simulated Co-Orders on Candidate Rank

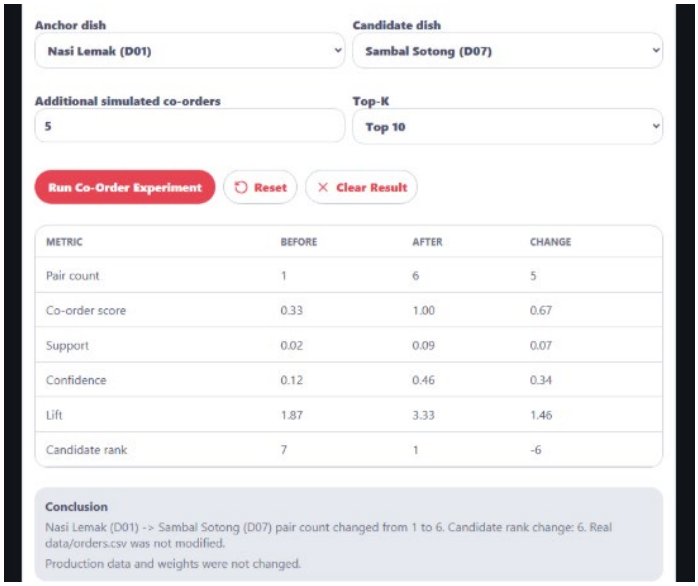


Figure 4.10: Co-Order History Impact Experiment Interface

These results answer RQ2 by showing that additional co-order evidence can strengthen measured associations and improve a candidate's ranking. At the same time, a stronger association does not always create visible rank movement when the dish is already ranked first.

4.4.3 Recommendation Method Comparison Results

The Recommendation Method Comparison addressed RQ3 by evaluating ingredient-only, co-order-only and fixed-hybrid ranking. Five historical baskets were selected from orders O056 to O060 and the final dish in each basket was hidden. The remaining dishes were then used as the known order context. The same liked preferences (rice and chicken) were applied in all five cases. No disliked ingredient was selected. A successful Hit@3 result was recorded when the hidden dish appeared within the first three recommendation positions.

Case	Historical order	Hidden dish	Ingredient-only rank	Co-order-only rank	Hybrid rank	Ingredient Hit@3	Co-order Hit@3	Hybrid Hit@3
MC-01	O060	Kuih Seri Muka (D06)	4	1	1	0	1	1
MC-02	O059	Gado-Gado (D29)	18	2	3	0	1	1
MC-03	O058	Pisang Goreng (D30)	23	1	1	0	1	1
MC-04	O057	Ketupat (D05)	1	2	1	1	1	1

MC-05	O056	Rojak Buah (D28)	25	3	3	0	1	1

Table 4.9: Hidden-Dish Recovery by Recommendation Method

The ingredient-only method recovered one hidden dish within the Top-3. The successful case was Ketupat in MC-04, which was placed at rank 1. In the remaining cases, the hidden dishes appeared at ranks 4, 18, 23, and 25. By comparison, the co-order-only method recovered all five hidden dishes. Three were placed first, one was placed second, and one was placed third. The fixed-hybrid method also recovered every hidden dish within the Top-3.

Recommendation method	**Successful Hit@3 cases**	**Hit@3 rate**	**Average hidden-dish rank**
Ingredient-only	1/5	20%	14.20
Co-order-only	5/5	100%	1.80
Fixed hybrid 0.4/0.6	5/5	100%	1.80

Table 4.10: Overall Recommendation Method Comparison

As shown in Table 4.10, co-order-only and fixed hybrid jointly achieved the highest Hit@3 rate of 100%. Both methods also produced an average hidden-dish rank of 1.80. In contrast, ingredient-only achieved a Hit@3 rate of 20% and an average rank of 14.20.

Despite having the same overall average, co-order-only and hybrid did not produce identical rankings in every case. For Gado-Gado in MC-02, co-order-only placed the hidden dish at rank 2, while hybrid placed it at rank 3. For Ketupat in MC-04, ingredient-only and hybrid placed the dish first whereas co-order-only placed it second.

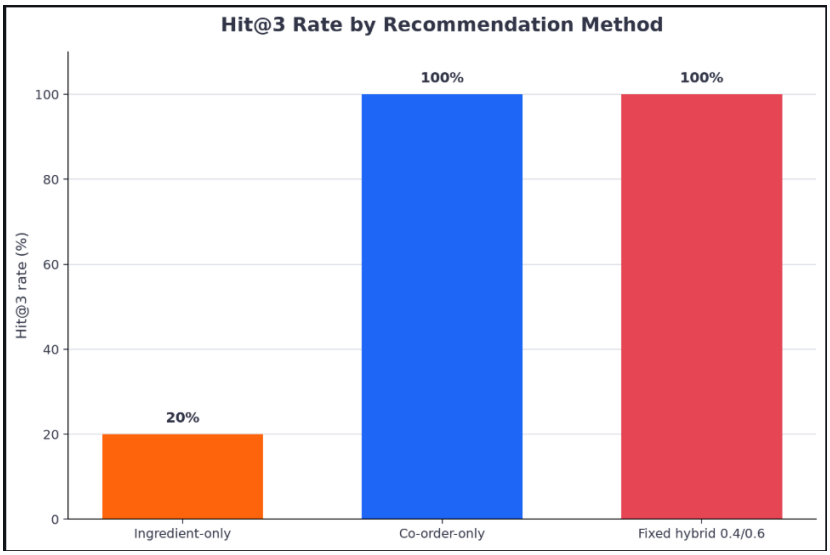


Figure 4.11: Hit@3 Rate by Recommendation Method

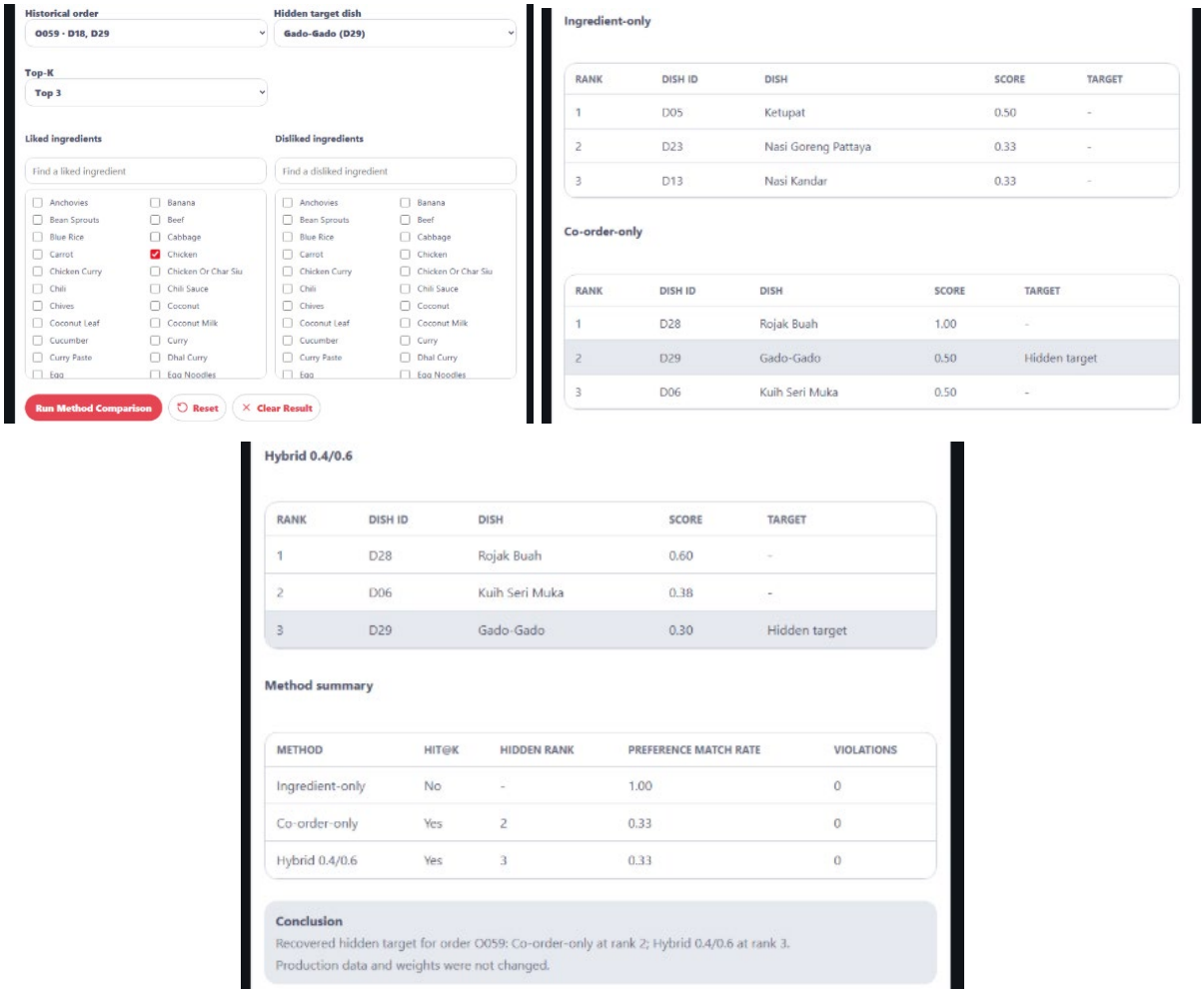


Figure 4.12: Hit@3 Recommendation Method Comparison Interface

The results answer RQ3 by demonstrating that, within the selected historical baskets, co-order evidence provided stronger hidden-dish recovery than ingredient matching alone. The fixed hybrid retained the successful recovery behaviour of the co-order method while also incorporating ingredient information.

4.4.4 Summary of Recommendation Results

The Ingredient Preference Impact experiment showed that explicit likes and dislikes affected recommendation results in different ways. Liked ingredients increased the positions of matching dishes while disliked ingredients removed conflicting dishes from the eligible list. Across the selected cases, no restriction violation was recorded.

The Co-Order History Impact experiment found that increasing the number of temporary co-orders strengthened pair count, support and association confidence. For Sambal Sotong, this additional evidence improved the candidate position from rank 7 to rank 1. Chicken Satay remained at rank 1 throughout because it already had a strong baseline relationship with Nasi Lemak.

Finally, the Recommendation Method Comparison produced a Hit@3 rate of 20% for ingredient-only and 100% for both co-order-only and fixed hybrid. These results suggest that historical basket relationships were particularly useful for recovering hidden dishes in the five selected cases.

Lastly, the results remain specific to the 30-dish menu, 60-order baseline and five selected hidden-dish scenarios. In addition, the method comparison was a controlled recovery demonstration rather than an independent train-and-test evaluation because the selected historical baskets remained part of the loaded history. A broader evaluation could remove each tested basket from the training history before generating its recommendation results.

4.5 Discussion

This section interprets the findings presented in Sections 4.3 and 4.4 in relation to the research questions, research objectives and previous studies reviewed in Chapter 2. Rather than repeating the numerical results, the discussion considers why the observed changes occurred, what they demonstrate about the recommendation approach and which conclusions cannot yet be drawn from the limited evaluation.

4.5.1 Effect of Ingredient Preferences

The findings for RQ1 show that explicit ingredient preferences can meaningfully alter the recommendation ranking even when no personal customer history is available especially selecting a liked ingredient increased the positions of dishes containing that ingredient. For example, Char Kway Teow moved substantially after bean sprouts was selected while rice-based dishes became more prominent when rice was chosen. This demonstrates that menu metadata can provide an immediate personalization signal without requiring the customer to have completed any previous orders.

This outcome is important within a single-restaurant environment because new customers naturally begin without individual ordering histories. Content-based recommendations can operate under this customer cold-start condition because it relies on item characteristics and stated preferences rather than previous user interactions. Food-recommendation research similarly recognizes that ingredients and other food attributes can provide useful domain information when behavioural evidence is unavailable or incomplete (Trattner & Elsweiler, 2017).

At the same time, the experiment revealed a limitation of simple ingredient matching. The degree of rank movement depended on whether the selected ingredient was represented within the dish metadata. A dish that did not contain rice or chicken could receive little support in the method-comparison experiment even if it formed a complement to the known order. Therefore, ingredient-based recommendation represents stated similarity but it does not necessarily capture which dishes customers commonly order together.

The disliked-ingredient results also have a different interpretation from ordinary rank movement. Pisang Goreng was removed entirely when banana was disliked even though it

originally occupied the highest baseline position. This confirms that the explicit restriction was not overridden by the dish's earlier rank or popularity. In practical terms, this behaviour is more appropriate than applying only a small score penalty because a clearly rejected ingredient should affect eligibility rather than merely reduce preference strength.

However, the result should not be interpreted as evidence of certified dietary. The exclusion depends on the accuracy and completeness of the recorded ingredient list and does not account for cross-contact, substitutions or undocumented preparation practices. Within this project, the zero restriction-violation result therefore demonstrates rule compliance under the available metadata but not medical safety.

Overall, RQ1 can be answered by concluding that liked ingredients raised compatible dishes within the ranking whereas disliked ingredients acted as hard exclusions. The findings support the use of explicit preferences as an interpretable response to customer cold start while also showing why ingredient matching alone cannot represent all forms of dish compatibility.

4.5.2 Influence of Co-Order Evidence

The results for RQ2 demonstrate that historical order baskets can provide a meaningful item-to-item behavioural signal. When temporary baskets containing Nasi Lemak and Sambal Sotong were introduced, the relationship between the two dishes became stronger and Sambal Sotong moved from a relatively weak position to the highest candidate rank. This indicates that repeated co-occurrence can allow a dish with limited initial evidence to become a prominent complementary recommendation.

The finding supports the decision to use implicit co-order behaviour rather than explicit customer ratings. In a restaurant ordering system, completed baskets are produced naturally through normal operations whereas ratings require customers to perform an additional feedback action. Item-based collaborative filtering is also recognized as a relevant approach for restaurant recommendation because it learns relationships among items rather than depending entirely on persistent customer profiles (Permana, 2024). Even so the Chicken Satay result shows that stronger evidence does not always produce visible rank movement. Chicken Satay was already ranked first at baseline, meaning that subsequent increases in pair count, support and association confidence

could not move it to a higher position. This represents a ceiling effect rather than a failure of the co-order signal.

The lift result requires further care. Although Chicken Satay appeared with Nasi Lemak more frequently after temporary baskets were added, its lift decreased slightly. This is not contradictory because lift considers the candidate's overall frequency as well as the pair frequency. Adding baskets containing Chicken Satay increased both its relationship with Nasi Lemak and its general prevalence in the temporary dataset. Hence, pair count, support, confidence, lift and rank should be considered together rather than treating one association measure as a complete description. From an operational perspective, these findings suggest that co-order recommendations may be useful after a customer has selected an anchor dish. The system can use relationships learned from completed baskets to identify possible accompaniments. By contrast, when no meaningful historical relationship exists, co-order evidence alone may provide limited information.

Therefore, RQ2 can be answered by concluding that additional co-order evidence strengthened the measured relationship between the selected dishes and could substantially improve candidate ranking. Still, the amount of rank movement depended on the candidate's existing position and its strength relative to other eligible dishes.

4.5.3 Comparison of Recommendation Methods

The results for RQ3 showed a clear difference between the three controlled recommendation configurations. Co-order-only and fixed hybrid recovered all five hidden dishes within the Top-3 whereas ingredient-only recovered one. Nevertheless, these results should not be interpreted as proof that ingredient-based recommendation is generally ineffective or that co-order recommendation is universally superior.

One reason for the difference is the structure of the experiment. Each hidden dish originated from an existing historical basket and the remaining dishes from that basket were supplied as context. This condition directly favoured methods that could use previously observed basket relationships. In contrast, the ingredient-only method received the same liked ingredients such as rice and chicken for all five cases. Hidden dishes such as Gado-Gado, Pisang Goreng and Rojak

Buah were not necessarily well represented by those selected preferences. As a result, their low ingredient-only rankings reflect a mismatch between the fixed preference inputs and the hidden targets rather than a general weakness of content-based recommendation. The Ketupat case provides evidence for this interpretation. Ketupat matched the selected rice preference and was ranked first by the ingredient-only method. In this scenario, content information was sufficiently aligned with the hidden dish and performed as well as better than the behavioural method. Thus, the relative usefulness of each approach depends on the evidence available in the current request.

Although the hybrid method achieved the same overall Hit@3 rate and average rank as co-order-only, it did not outperform co-order-only in every case. For example, co-order-only ranked Gado-Gado second whereas hybrid ranked it third. This occurred because the hybrid scores also incorporated ingredient information that did not strongly support the hidden dish. Conversely for Ketupat, the hybrid method benefited from both the contextual and ingredient signals and placed the dish first. Therefore, the principal benefit of hybridization in this experiment is not a demonstrated increase beyond the best individual method. Instead, it is the ability to retain two different sources of relevance. Collaborative evidence captures behavioural dish relationships while ingredient evidence represents explicit customer preferences. Hybrid food-recommendation approaches are commonly motivated by this complementary relationship between content and collaborative information rather than by an assumption that one fixed combination must dominate every individual case (Singh & Dwivedi, 2023).

RQ3 can be answered by concluding that co-order-only and fixed hybrid provided stronger hidden-dish recovery than ingredient-only under the selected basket-based scenarios. However, the case-level differences show that no single method should automatically receive the same importance under every evidence condition. This supports the production system's use of adaptive weighting, while the fixed configurations remain useful for controlled comparison.

4.5.4 Relationship to Previous Research and Practical Implications

The findings are consistent with the main research gap identified in Chapter 2. Ingredient information provided recommendations without requiring customer history while co-order data captured relationships that ingredient similarity alone could not represent. Their combination offers a practical way of supporting personalization in a limited-data restaurant environment. The

results also illustrate why food recommendation differs from recommending simpler media items. Food selection can depend on ingredients, combinations of dishes, dietary restrictions, context, familiarity and individual preference. Reviews of food recommender systems highlight this specialized and multidimensional character as well as continuing challenges involving data availability, context and evaluation quality (Trattner & Elsweiler, 2017).

For a small restaurant, the results suggest three practical uses. Explicit preferences can provide immediate guidance to new customers. Co-order evidence can support complementary suggestions after a dish has been selected and the hybrid system can use both signals when they are available. Moreover, completed orders can gradually expand the behavioural evidence without requiring customers to submit ratings. The findings also support the inclusion of explanations and evidence indicators. Since a dish may rank highly for different reasons, customers and administrators should be able to distinguish a strong ingredient match from repeated co-order support or a popularity-based fallback. Explainable-recommendation research similarly emphasises that explanations can help users and system designers understand why an item was recommended rather than receiving only an unexplained output (Zhang & Chen, 2020).

4.5.5 Limitations

The evaluation contains several limitations that affect the strength and generalizability of its conclusions. First, the experiments used a menu of 30 dishes and a baseline of 60 historical baskets. The co-order relationships may therefore be sensitive to individual records and may not reflect the behaviour of a restaurant with a substantially larger, more varied or continuously changing customer population.

Second, only three ingredient scenarios, two co-order pairs and five hidden-dish cases were examined. Although these cases are sufficient to show the implemented behaviour but are not large enough for statistical comparison or a broad claim about recommendation accuracy.

Third, the same liked ingredients were used across all five method-comparison cases. This provided consistency but it also favoured dishes related to rice or chicken and disadvantaged hidden dishes with unrelated ingredients. A wider evaluation should repeat the comparison using

several preference profiles, including neutral conditions and preferences selected specifically from each basket.

Despite these limitations, the evaluation fulfils its intended purpose: it demonstrates that ingredient preferences, co-order evidence and fixed recommendation configurations produce observable and repeatable differences within the implemented prototype. Thus, the conclusions are restricted to the behaviour of the system under the stated controlled conditions.

Chapter 5 Conclusion and Improvement

5.1 Overview

This chapter concludes the study by consolidating its main findings, contributions, limitations and future development directions. Section 5.2 evaluates the achievement of the research aim and objectives. Section 5.3 presents the study's contributions, Section 5.4 proposes directions for future work, and Section 5.5 concludes the study.

5.2 Achievement of the Research Aim and Objectives

The overall aim of this study was to develop and evaluate a lightweight, explainable and personalized restaurant ordering system for a single-restaurant environment with limited historical data. Based on the completed implementation and evaluation, the research aim was achieved within the defined scope of an academic prototype.

Research objective	Status	Basis of achievement
RO1: Develop a mobile-first web-based restaurant ordering system	Achieved	A complete customer and administrator workflow was implemented and successfully tested.
RO2: Implement ingredient-based recommendation with preference restrictions	Achieved	The system uses liked ingredients to support ranking and disliked ingredients to exclude unsuitable dishes.
RO3: Implement co-order-based item-to-item collaborative filtering	Achieved	Historical order baskets are used to identify behavioural relationships between dishes without requiring explicit ratings.
RO4: Integrate recommendation signals through an explainable hybrid approach	Achieved	Ingredient, co-order, popularity, and contextual signals are combined, with supporting reasons and evidence presented for recommendations.
RO5: Develop controlled and non-destructive recommendation experiments	Achieved	Three experiment modules were implemented without modifying the operational order history.
RO6: Evaluate the behaviour of the recommendation approaches	Achieved	The implemented methods were evaluated using ranking changes, association measurements, restriction compliance, hidden-dish rank, and Hit@K.

Table 5.1 summarizes the achievement of each research objective

At the system level, the prototype demonstrated that personalized recommendation could be incorporated into a functioning restaurant-ordering workflow. At the recommendation level, the project successfully implemented explicit preference processing, behavioural co-order analysis, and hybrid scoring. From an evaluation perspective, the controlled tools provided repeatable evidence of how these recommendation signals affected the output. All in all, those research objectives were achieved within the intended prototype scope.

5.3 Contributions of the Study

The contribution of this study is primarily artefact oriented. Design Science Research recognizes that a computing study may contribute through the construction of a useful artefact, the knowledge generated through its design or an appropriate balance between artefact and theory (Baskerville et al., 2018). In this study, established recommendation techniques were adapted into a working solution for a limited-data, single-restaurant setting rather than used to propose a new mathematical model.

First, the study contributes to a **context-specific computing artefact** that demonstrates the feasibility of incorporating personalization into a complete restaurant-ordering environment. The prototype brings customer ordering, administrative processing, recommendation generation and evaluation into one web-based application. Its significance lies in adapting existing recommendation concepts to the operational and data constraints of a small restaurant. This context-specific approach is relevant because food recommender systems differ substantially in their intended users, available data, recommendation goals, implementation techniques and evaluation methods (Bondevik et al., 2024).

Beyond the completed artefact, the study contributes a **transparent separation of recommendation decisions**. The system distinguishes whether a dish is eligible, how eligible dishes are ranked and how much evidence supports each recommendation. Besides, explicit restrictions are considered before ranking while evidence confidence remains separate from the final recommendation score. This separation offers a clear design structure for recommendation systems where restrictions, ranking signals and explanations serve different purposes. It also aligns

with explainable-recommendation principles that emphasize presenting understandable reasons for system outputs rather than relying solely on unexplained scores (Zhang & Chen, 2020).

A further contribution is the **non-destructive evaluation approach** embedded within the administrator environment. The study developed controlled tools that allow individual recommendation signals and fixed method configurations to be examined without modifying genuine historical-order data. This separation allows the same artefact to support both normal ordering activities and repeatable academic evaluation while preserving the integrity of its operational evidence.

Collectively, these contributions are application-focused rather than algorithmically focused. The study contributes a functioning artefact for a specific restaurant context, a transparent structure separating eligibility, ranking and supporting evidence and an evaluation environment that isolates simulated conditions from operational data. These elements provide a practical foundation for further investigation of ***lightweight and explainable personalization in small restaurant environments.***

5.4 Future Work

For future work, the project should move beyond controlled prototype evaluation towards a more realistic restaurant environment. The proposed directions focus on strengthening evaluation validity, improving operational readiness, extending recommendation capability and assessing the system with actual users.

To begin with, future evaluations should use a larger historical order dataset and a stricter offline testing procedure. In detail, the complete basket selected for testing should be removed from the recommendation history before its hidden dish is predicted. A chronological split could also be applied so that each prediction uses only interactions recorded before the test order. These procedures would reduce the possibility of information leakage and produce a more realistic estimate of recommendation performance (Ji et al., 2023). Moreover, the evaluation could be repeated across a greater number of baskets and preference profiles using additional measures such as ***Mean Reciprocal Rank*** and ***Normalized Discounted Cumulative Gain***.

Beyond offline evaluation, a study involving real customers and restaurant staff would provide evidence that cannot be obtained from automated testing alone. Customer testing could examine task completion, decision time, recommendation acceptance, perceived relevance and the usefulness of recommendation explanations. At the same time, restaurant staff could evaluate whether the administrator interface and recommendation insights are understandable and operationally useful. Since food recommender systems may require both technical and user-centred evaluation which combine behavioural results with participant feedback would provide a more complete assessment of the system (Bondevik et al., 2024). Likewise, explanation quality should be assessed directly because the presence of an explanation does not necessarily mean that users understand or trust it (Zhang & Chen, 2020).

From a software engineering perspective, the local file-based persistence should eventually be replaced with a relational database. This change would support transactional order updates, durable sessions, concurrent access, structured backup and more reliable recovery following application failure. In addition, the authentication model could be expanded to include individual staff accounts, securely hashed passwords, role-based permissions, CSRF protection, rate limiting and HTTPS deployment. Collectively, these improvements would prepare the system for a controlled restaurant trial rather than limiting it to a local demonstration environment.

At the recommendation level, future versions could investigate how preferences change across repeated visits and different dining contexts. Once a larger volume of genuine interaction data becomes available, time-aware or sequence-based methods could be compared with the current lightweight approach. Sequence-based food recommendation research indicates that ordered interactions can help represent both longer-term preferences and recent behavioural changes (Zhang et al., 2023). Nevertheless, any more complex method should be evaluated against the existing transparent baseline to determine whether its additional computational cost produces a meaningful improvement.

Finally, the quality of the menu metadata could be strengthened through standardized ingredient names, verified dietary labels, allergen fields, preparation methods and portion information. These additions could improve filtering precision and make customer warnings more informative. However, any function related to allergy management would require formal restaurant verification and clear data-governance procedures before it could be presented as a safety feature.

In conclusion, these directions provide a pathway from a prototype towards a more robust and practically evaluated restaurant system. They are intended to strengthen the reliability, usability, and deployability of the solution while preserving its lightweight and explainable design.

5.5 Conclusion

Overall, this study achieved its aim of developing and evaluating a personalised restaurant ordering system suited to a limited-data, single-restaurant environment. The completed artefact provides evidence that lightweight recommendation can operate as part of a practical ordering process while remaining understandable and manageable within the defined prototype scope.

More broadly, the study demonstrates that personalization does not necessarily require large customer profiles, explicit ratings or computationally intensive models. Instead, available menu information and historical ordering behaviour can be organized into a transparent decision-support mechanism that complements the restaurant-ordering workflow.

Nevertheless, the conclusions are restricted to the controlled conditions under which the prototype was developed and evaluated. They should be understood as evidence of technical feasibility rather than proof of production readiness or commercial effectiveness.

Ultimately, the project establishes a practical basis for further work on explainable and resource-efficient restaurant recommendation. It shows that meaningful personalization can be introduced into a small restaurant system while preserving clarity, operational simplicity, and customer choice.

References

Baskerville, R., Baiyere, A., Gregor, S., Hevner, A., & Rossi, M. (2018). Design science research contributions: Finding a balance between artifact and theory. *Journal of the Association for Information Systems, 19*(5), 358–376. https://doi.org/10.17705/1jais.00495

Biega, A. J., Potash, P., Daumé III, H., Diaz, F., & Finck, M. (2020). *Operationalizing the legal principle of data minimization for personalization*. Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 399–408.

Bjarnason, E., Lang, F., and Mjöberg, A. (2023). An empirically based model of software prototyping: A mapping study and a multi-case study. *Empirical Software Engineering, 28*, Article 115. https://doi.org/10.1007/s10664-023-10331-w

Bondevik, J. N., Bennin, K. E., Babur, Ö., & Ersch, C. (2024). A systematic review on food recommender systems. *Expert Systems with Applications, 238*, 122166. https://doi.org/10.1016/j.eswa.2023.122166

Chow, Y.-Y., Haw, S.-C., Naveen, P., Anaam, E. A., & Bin Mahdin, H. (2023). Food recommender system: A review on techniques, datasets and evaluation metrics. *Journal of System and Management Sciences, 13*(5), 153–168. https://doi.org/10.33168/JSMS.2023.0510

Gao, X., Feng, F., He, X., Huang, H., Guan, X., Feng, C., Ming, Z., & Chua, T.-S. (2020). Hierarchical attention network for visually-aware food recommendation. *IEEE Transactions on Multimedia, 22*(6), 1647–1659. https://doi.org/10.1109/TMM.2019.2945180

Gunawardena, D., & Sarathchandra, K. (2020). BestDish: A digital menu and food item recommendation system for restaurants in the hotel sector. In *2020 International Conference on Image Processing and Robotics (ICIP)* (pp. 1–7). IEEE. https://doi.org/10.1109/ICIP48927.2020.9367357

Ji, Y., Sun, A., Zhang, J., & Li, C. (2023). A critical study on data leakage in recommender system offline evaluation. *ACM Transactions on Information Systems, 41*(3), Article 75, 1–27. https://doi.org/10.1145/3569930

Keshav, A., Viswanathan, S., Dinesh, R., & Balamurugan, M. (2023). Smart Dine-in: A personalized food recommendation system. In *2023 Intelligent Computing and Control for Engineering and Business Systems (ICCEBS)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICCEBS58601.2023.10448742

Li, X., Jia, W., Yang, Z., Li, Y., Yuan, D., Zhang, H., & Sun, M. (2018). Application of intelligent recommendation techniques for consumers' food choices in restaurants. *Frontiers in Psychiatry, 9*, 415. https://doi.org/10.3389/fpsyt.2018.00415

Munaji, A. A., & Emanuel, A. W. R. (2021). Restaurant recommendation system based on user ratings with collaborative filtering. *IOP Conference Series: Materials Science and Engineering, 1077*(1), 012026. https://doi.org/10.1088/1757-899X/1077/1/012026

National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1. doi:10.6028/NIST.AI.100-1

Permana, K. E. (2024). Comparison of user based and item based collaborative filtering in restaurant recommendation system. *Mathematical Modelling of Engineering Problems, 11*(7), 1922–1928. https://doi.org/10.18280/mmep.110723

Sharma, K., Mandapati, K. V., Pattekar, M. C., Kumar, K. K. T., & Srinivasa, G. (2023). Food recommendation system based on collaborative filtering and taste profiling. In *2023 3rd International Conference on Intelligent Technologies (CONIT)* (pp. 1–6). IEEE. https://doi.org/10.1109/CONIT59222.2023.10205379

Shinagam, S. (2025). *Comparison of collaborative filtering and content-based filtering for recommendation systems* [Master's thesis, Utrecht University]. Utrecht University Student Theses Repository. https://studenttheses.uu.nl/handle/20.500.12932/50247

Singh, R., & Dwivedi, P. (2023). Food recommendation systems based on content-based and collaborative filtering techniques. In *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE. https://doi.org/10.1109/ICCCNT56998.2023.10307080

Tian, Y., Zhang, C., Metoyer, R., & Chawla, N. V. (2022). Recipe recommendation with hierarchical graph attention network. *Frontiers in Big Data, 4*, 778417. https://doi.org/10.3389/fdata.2021.778417

Trattner, C., & Elsweiler, D. (2017). Food recommender systems: Important contributions, challenges and future research directions. *arXiv*. https://doi.org/10.48550/arXiv.1711.02760

United Nations Department of Economic and Social Affairs. (n.d.-a). *Goal 9: Industry, innovation and infrastructure*. United Nations Sustainable Development Goals. Retrieved July 27, 2026, from https://sdgs.un.org/goals/goal9

United Nations Department of Economic and Social Affairs. (n.d.-b). *Goal 12: Responsible consumption and production*. United Nations Sustainable Development Goals. Retrieved July 27, 2026, from https://sdgs.un.org/goals/goal12

Zhang, J., Wang, Z., Liu, W., Liu, X., & Zheng, Q. (2023). A unified approach to designing sequence-based personalised food recommendation systems: Tackling dynamic user behaviours. *International Journal of Machine Learning and Cybernetics, 14*, 2903–2912. https://doi.org/10.1007/s13042-023-01808-7

Zhang, Y., & Chen, X. (2020). Explainable recommendation: A survey and new perspectives. *Foundations and Trends in Information Retrieval, 14*(1), 1–101. https://doi.org/10.1561/1500000066

Zhang, Y., Zhou, X., Meng, Q., Zhu, F., Xu, Y., Shen, Z., & Cui, L. (2025). *Multi-modal food recommendation using clustering and self-supervised learning* (Version 2) [Working paper]. arXiv. https://doi.org/10.48550/arXiv.2406.18962